



Agentic Much? Adoption of Coding Agents on GitHub

Romain Robbes, Théo Matricon, Thomas Degueule, Andre Hora, Stefano Zacchiroli

► To cite this version:

Romain Robbes, Théo Matricon, Thomas Degueule, Andre Hora, Stefano Zacchiroli. Agentic Much? Adoption of Coding Agents on GitHub. ACM Transactions on Software Engineering and Methodology, 2026, pp.1-43. <10.1145/3822180>. <hal-05655264>

HAL Id: hal-05655264

<https://hal.science/hal-05655264v1>

Submitted on 12 Jun 2026

HAL is a multi-disciplinary open access archive for the deposit and dissemination of scientific research documents, whether they are published or not. The documents may come from teaching and research institutions in France or abroad, or from public or private research centers.

L'archive ouverte pluridisciplinaire **HAL**, est destinée au dépôt et à la diffusion de documents scientifiques de niveau recherche, publiés ou non, émanant des établissements d'enseignement et de recherche français ou étrangers, des laboratoires publics ou privés.



Copyright - All rights reserved

Agentic Much? Adoption of Coding Agents on GitHub

ROMAIN ROBBES, Univ. Bordeaux, CNRS, France

THÉO MATRICON, Univ. Rennes, Inria, CNRS, IRISA, France

THOMAS DEGUEULE, Univ. Bordeaux, CNRS, France

ANDRE HORA, UFMG, Brazil

STEFANO ZACCHIROLI, Institut Polytechnique de Paris, France

In the first half of 2025, *coding agents* have emerged as a category of development tools that have very quickly transitioned to the practice. Unlike “traditional” code completion LLMs such as Copilot, agents like Cursor, Claude Code, or Codex operate with high degrees of autonomy, up to generating complete pull requests starting from a developer-provided task description. This new mode of operation is poised to change the landscape in an even larger way than code completion LLMs did, making the need to study their impact critical. Also, unlike traditional LLMs, coding agents tend to leave more explicit traces in software engineering artifacts, such as co-authoring commits or pull requests. We leverage these traces to present the first large-scale study (128,018 projects) of the adoption of coding agents on GitHub, finding an estimated adoption rate of 22.20%–28.66% on February 21st, 2026, which is very high for a technology only a few months old—and increasing. We carry out an in-depth study of the adopters we identified, finding that adoption is broad: it spans the entire spectrum of project maturity; it includes established organizations; and it concerns diverse programming languages or project topics. At the commit level, we find that commits assisted by coding agents are larger than commits only authored by human developers, and have a large proportion of features and bug fixes. These findings highlight the need for further investigation into the practical use of coding agents.

Additional Key Words and Phrases: Coding Agents, AI4SE, Large Language Models, Software Repositories

ACM Reference Format:

Romain Robbes, Théo Matricon, Thomas Degueule, Andre Hora, and Stefano Zacchiroli. 2026. Agentic Much? Adoption of Coding Agents on GitHub. 1, 1 (June 2026), 43 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

In the span of a year, coding agents such as Claude Code, Codex, or OpenCode (among many others) have transitioned from promising tools in research papers to practical products seeing practical adoption. While the emergence of LLMs in programming, such as earlier versions of Copilot, has been and is still the subject of extensive study, coding agents, even if they leverage the same underlying technology, are markedly different. What distinguishes coding agents is both the scale of the tasks that they undertake and the degree of autonomy with which they undertake these tasks. Developers can use a traditional LLM to accelerate their programming by completing lines or blocks of code. However, they can (and do—see Section 10) *delegate entire tasks* to coding agents, such as finding and fixing bugs, or implementing new features. Depending on their workflow,

Authors’ Contact Information: [Romain Robbes](mailto:romain.robbes@labri.fr), romain.robbes@labri.fr, INP, LaBRI, UMR 5800, Univ. Bordeaux, CNRS, Bordeaux, France; [Théo Matricon](mailto:theo.matricon@inria.fr), theo.matricon@inria.fr, Univ. Rennes, Inria, CNRS, IRISA, Rennes, France; [Thomas Degueule](mailto:thomas.degueule@labri.fr), thomas.degueule@labri.fr, INP, LaBRI, UMR 5800, Univ. Bordeaux, CNRS, Bordeaux, France; [Andre Hora](mailto:andrehora@dcc.ufmg.br), andrehora@dcc.ufmg.br, Department of Computer Science, UFMG, Belo Horizonte, Brazil; [Stefano Zacchiroli](mailto:stefano.zacchiroli@telecom-paris.fr), stefano.zacchiroli@telecom-paris.fr, LTCI, Télécom Paris, Institut Polytechnique de Paris, Palaiseau, France.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

© 2025 Copyright held by the owner/author(s).

ACM XXXX-XXXX/2026/6-ART

<https://doi.org/XXXXXXX.XXXXXXX>

coding agents can operate under close supervision (with a developer manually approving each action), up to nearly complete autonomy, e.g., acting on a bug report and submitting a complete pull request for review.

The ability to delegate tasks to a coding agent in such a fashion holds great promise. However, the few human studies on coding agents show contrasting results [3, 17]. We detail the working of coding agents in Section 2 and the related work on empirical studies of coding assistants in Section 3.

The paucity of work studying coding agents motivates us to study this from another angle. While completion-based coding assistance embedded in the IDE is hard to track in the wild, since it leaves no explicit traces, coding agents leave abundant traces in software repositories, in the form of configuration files, summaries of knowledge they acquired, or even explicit metadata (e.g., author or co-author information) in commits and pull requests [28]. This paves the way to study the adoption of coding agents in the real world at a very large scale, a point of view that is highly complementary to the existing qualitative studies [32]. We detail our methodology, based on the analysis of software artifacts on a very large scale, in Section 4.

We find that the adoption of coding agents has been very rapid. As of February 21st, 2026, in a sample of 128,018 projects, 12.08% of the projects present traces of adoption at the file level; a further 11.51% project show adoption at the commit level leading us to estimate a total adoption of 22.20%, with a high estimate of up to 28.66% (See Section 5)¹.

Beyond quantifying adoption in a binary way, we conduct more detailed analyses to quantify the extent of adoption and its distribution over the project maturity spectrum. We find a bias towards younger projects; beyond that, and somewhat unexpectedly, we find that larger and more mature projects adopt coding agents in comparable proportions to smaller and less mature ones. We also find that a sizeable proportion of projects have adopted coding agents in a pervasive manner, with a small, but non-negligible set of extreme adopters. We detail these findings in Section 6.

Going further, we study the adoption in more specific contexts; finding adoption in diverse contexts such as established organizations, types of projects as described by their GitHub topics, and programming languages (Section 7). We study the evolution of the adoption over time and across tools and organizations, observing a rapid growth since March 2025, and identify that a small number of coding agents account for the majority of adoption (Section 8). Finally, we study the contributions authored or co-authored by coding agents at the commit level, finding that they are larger than those authored by humans only (Section 9); we perform a manual analysis of the content of a sample of commits, to find the types of tasks that are performed with the help of coding agents, finding it is concentrated in *features* and *fixes* (Section 10).

The current extent and the rapid increase of the adoption of coding agents have several implications, both in practice and in research (notably, there are ample avenues for future work). Our study has limitations: in particular, we discuss in which ways our heuristics may lead to overcounts or undercounts of the adoption. We discuss the implications, limitations, and future work related to our study in Section 11.

2 Background

2.1 Coding Agents

The agentic loop. We use the following definition for coding agents: a *coding agent* is a LLM running in a loop with tool access that aims to complete a given goal. The tools enable the LLMs to have rich interactions with its environment. The LLM loop, which we call *agent loop*, is simple: the

¹These numbers reflect the version of the study when it was accepted. We periodically update this study in our online ArXiv report [29]; as of the end of May 2026, we estimate the adoption to be between 31.63% and 38.23%.

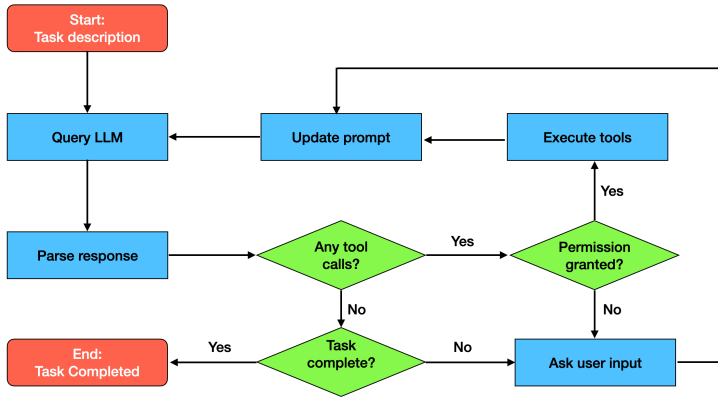


Fig. 1. Typical workflow of a coding agent

LLM proposes an action, gets feedback, and then use the feedback to offer another action, more precisely:

- (1) query the LLM with a sequence of interactions, initially this is just the task description;
- (2) parse the response given by the LLM for the presence of tool usages and task completion;
- (3) if tool calls are present: optionally ask the user for permission before using sensitive tools;
- (4) if task is not completed: add the LLM's response to the sequence of interactions, replacing tool calls with their results.

Figure 1 shows a graphical depiction of this process. The agent notifies the user when it believes it has completed the task, ending the loop. Some agents go as far as to commit the changes to the repository, or author a pull request. Depending on its configuration, the agent may insert co-authorship information at this point (e.g., `Co-authored-by: copilot-swe-agent`).

The biggest contributor to the performance of coding agents is the LLM. There has been rapid progress of LLMs towards producing structured output or improved tool usage, which are key for agentic use. Furthermore, there has been a recent growth in the reasoning capabilities of agents over long sequence of interactions which is a key driving factor for the performance of coding agents.

LLM Harness and tool use. Tools access boosts coding agents with more reliable capabilities; the LLM's *harness* connects the LLM to these tools. The harness provides the list of available tools to the LLM as part of its prompt. Part of the LLM's training involves tool use, which, from the LLM's point of view, consists in outputting a tool call using a structured format such as JSON. The harness will query the LLM, and parses the LLM's output in order to detect tool calls. It then invokes the tools on behalf of the LLM, and feeds the output of the tool back to the LLM. Depending on the degree of autonomy desired, the harness may ask the user for confirmation before executing certain tools (e.g., to review code edits, or to approve shell commands).

Tools and context. Software projects constitute a large context for an LLM, which has a limited context window. For traditional coding assistants, context is often pre-calculated by the IDE, and included to the LLM's prompt. Coding agent allow the same functionality; they also allow developer to manually specify relevant context by mentioning specific files. However, a coding agent can also augment its context: tool calls enable dynamic and interactive queries. This enable coding agents to perform more autonomous workflows, such as:

- Adapt its context to the task at hand, by using tool calls to explore the code base dynamically;
- Automatic debugging of the code thanks to compilation and a potential suite of tests, using the feedback given from these tools the agent can propose solutions.

There is a large variety of tools that agents can use. Some coding agents can run arbitrary shell commands in a terminal (this defines a small set of tools, but allows great flexibility), whereas other operates in more controlled environments that restrict the commands they can run. Some specific tools enable browsing the web to gather information available. Other tools allow coding agents to run a project test suite; this is highly valuable for the agent to gain feedback on their work. The specific tools available are user dependent; emerging standards such as the Model Context Protocol (MCP) ease the definition of new tools.

Given a task (e.g., “implement feature X and make sure the tests pass”), the agent will execute the LLM in a loop until it determines that the task is completed. In the feature implementation example, the agent could start by exploring the code base to find relevant code, then edit the code until it compiles (processing errors from the compiler), and then iterate until tests pass (if they exist; the agent may also be tasked to write tests in the first place).

Autonomy and oversight. Notably, this process can be lengthy, as the LLM is used multiple times, processing inputs of increasing size (often growing to tens or hundreds of thousands of tokens). Agents tend to have an asynchronous mode of interaction: they do their work in the background, only stopping when the task is done, or when they require input from the developer. Developers can tend to the agents in between their other tasks. Depending on the degree of autonomy desired, the developer may interrupt the process at any point and provide some guidance to the LLM. Particularly trusting users (or careful developers using sandboxes and extensive test suites) may provide high or full autonomy to the LLM, setting the harness to auto-accept any tool the LLM desires.

Developers may provide guidance to the agent by providing more detailed instructions (e.g., “implement feature X in module Y, using library Z”), which will reduce the need for searches, or give feedback on the work (e.g., “this code does not conform to the project’s convention, always do X, not Y”). The LLM may also compile this guidance in the form of instructions in text files; with appropriate conventions, the harness will automatically add these files to the LLM’s context window in subsequent iterations. The footnote below links to two examples of such files containing guidance about specific projects, describing their high-level organization, commands to interact with the project, conventions to follow, among others.² As such knowledge is valuable, these files are often committed to the repository, so that the agent can consult them overtime, and, if necessary, update them. In this study, we look for the presence of these *guidance files* as markers for the use of LLMs, together with other explicit markers in commits and pull requests.

2.2 Traces and heuristics

Coding agents often leave visible traces of their use. These traces can be in the form of a specific pattern in commit messages or metadata, a branch name, or the presence of a specific file in a repository. To detect traces of coding agents activity in this work, we rely on our previous work [28] in which we produced an extensive list of heuristics about the traces left by more than 50 different coding agents. These heuristics were validated with the data found on the GitHub platform. Heuristics are, by definition, not perfect, but we ensure that they do not produce many false positives. We provide below a short overview of the traces left by coding agents. Each agent has a specific workflow, and may leave a subset of these traces.

²Example 1 and Example 2; the next section has additional examples.

2.2.1 Files. Files can be used to configure coding agents and add specific instructions for them. We distinguish the following file types used to configure agents behavior: *Configuration files*, *Rules*, and *Guidance files*. Configuration files allow to choose between predefined options of the agent in a structured language such as YAML, whereas rules and guidance files enable guiding the agent with custom instructions in natural language.

Configuration files indicate that the agent has been set up within the repository. They contain the most important options for configuring the agent's behavior and thus can offer insight into how the agents work, such as permissions, i.e., the actions the agent can take without waiting for the user's permission. These settings are highly tool-specific. Some settings influence the presence or absence of other traces, hindering the detection of agents (the Peril of Partial Observability [28]). For instance, Claude Code's `.claude/settings.json` has an option to disable traces left in commits, while Codex can change the default prefix it uses to create branches.

Rules and Guidance files aim to guide the agent through natural language specifications that the agent should follow. They are generally written in markdown format and are added to the user prompt before sending it to the LLM. They can have large or specific scopes. They are sometimes generated by an agent. Their goal is to encode *tacit knowledge* that is necessary to the agent to tackle the tasks; this is why they are often shared among developers and most of the time committed to the repository. Rules are context specific³ whereas guidance files are broader.⁴ Guidance files may contain detailed task descriptions that highlight the steps to solve a specific task⁵ or tactics and strategies the LLM should use to tackle problems in general.⁶

For example, the guidance file `AGENTS.md`⁷, originally used by Codex, is a simple open format for guiding coding agents and is adopted by several agents, including Jules, Cursor, Aider, and Copilot. Claude uses `CLAUDE.md`, as stated in its documentation: "*CLAUDE.md is a special file that Claude automatically pulls into context when starting a conversation. This makes it an ideal place for documenting: common bash commands, core files and utility functions, code style guidelines, testing instructions [...]*".⁸

2.2.2 Commits. Most agents are given the capability to commit their work automatically or given a tool to do so, for example via the shell. If allowed to commit, the agent should also write the commit message. Within commits, agents can advertise their contribution to the commit through different mechanisms:

- By adding themselves as a *co-author* of the commit, with the agent's user advertised as the main author. In this case, they use the "Co-authored-by:" trailer, which is added to the commit. Of note, agents may not adhere to the convention exactly, occasionally using different casings.⁹
- Less commonly, agents can be recorded as commit authors, similarly to how bots such as dependabot operate. To do so they provide a standard name and email address in commit metadata like Git's "author" and/or "committer" fields.¹⁰

³Example cursor rules with technology stack, programming principles, style issues, and how to test (27 lines).

⁴Example high-level project description with: overview; key features; build, launch, test, and lints commands; project architecture (main modules and classes); development workflow; and typical tasks (239 lines)

⁵Partially executed implementation plan for a large refactoring, with: completed tasks; pending tasks by priority; recommended execution plan; issues to watch for; success criteria; testing strategy; references (508 lines)

⁶Example guidance with strategy, for clarifying specifications that are under-specified (176 lines)

⁷<https://agents.md>, listing 24 agents using the format on 10/06/26

⁸<https://www.anthropic.com/engineering/claude-code-best-practices>

⁹Example commit, co-authored by Claude Code, fixing a bug and adding tests (two files changed, +75/-2 lines).

¹⁰Example commit, authored by Copilot, adding a feature and tests (16 files in 13 directories changed, +380/-48 lines)

- Some agents also add additional trailer pseudo-headers to the commit message, e.g., “Generated by: Claude”.

All these traces can be explicitly disabled via configuration files, hiding the participation of agents.

2.2.3 Users, Issues, and Pull requests. Specific user accounts can be linked to coding agents, i.e., the activity of those users are in fact agent activities. This enables users to assign reviews or issues to specific coding agents as users.¹¹ Issues can then be used as the descriptions of tasks that are then delegated to coding agents. Some agents, when assigned such an issue or a more general task, will work via Pull Requests (PRs), in which they may author multiple commits depending on the task scope. Developers can interact with the agent via the GitHub interface, for example, delegating additional tasks.¹² A PR provides more information than a commit, with the whole trace of the development and the interaction with GitHub Actions, users, and reviewers. The complexity of PRs authored by coding agents can range from the quite simple to authoring complex new features from scratch.¹³

3 Related work

3.1 Studies of LLM-based coding assistants

Controlled experiments. Experiments show mixed results on productivity: Imai [16] found Copilot improved productivity but reduced code quality versus human pair programming, while Vaithilingam et al. [36] reported more task failures due to incorrect Copilot suggestions despite user preference. Peng et al. [27], however, found 95 programmers completed HTTP server implementations 55% faster with Copilot.

Observation studies. Barke et al. [2] identified two primary usage modes for developers: *acceleration*—where developers with clear goals prefer small, quickly-validated suggestions to maintain flow—and *exploration*—where developers lacking clear plans (e.g., with unfamiliar APIs) favor larger suggestions as starting points. Mozannar et al. [23] found developers spent 22% of time evaluating Copilot suggestions versus only 14% writing new code, indicating that acceptance rates mask the true cognitive effort of AI-assisted programming.

Surveys and interviews. Liang et al. [21] surveyed 410 developers, identifying successful use cases (boilerplate, proof of concepts) and barriers (validation time, non-functional requirements). Wang et al. [37] studied trust in AI assistants through interviews with developers based on real-world Copilot usage.

Telemetry. Murali et al. [24] reported Meta’s CodeCompose achieved a 22% acceptance rate and generated 8% of company-wide code. Among 2,000 Copilot users, Ziegler et al. [40] found acceptance rate best predicted perceived productivity.

Usage in software repositories. Tufano et al. [34] analyzed ChatGPT usage traces in 467 GitHub instances, developing a taxonomy of 45 software engineering tasks including feature implementation, documentation, software quality, and development processes. Xiao et al. [38] analyzed over 1,200 explicit generative AI traces from ChatGPT and Copilot in software artifacts, focusing on targeted tasks and churn changes before and after adoption. They found no significant change in churn post-adoption. On the other hand, the white papers from GitClear [11, 12] report increased churn and code duplication after Copilot’s release over time. However, their lack of usage traces

¹¹Example coding agent account for OpenHands

¹²Example PR with user delegating additional tasks to OpenHands (improving documentation and code consistency)

¹³Example PR with user interacting with Copilot to implement a feature (3 follow-ups, 8 commits, +350/-15 lines)

prevents distinguishing actual Copilot use, limiting the analysis to a rough approximation that may not exclude other confounding factors.

3.2 Studies of Coding Agents

Experiments and observation. Becker et al. [3] found that developers using Cursor overestimated productivity gains by 20% on average, while effective productivity actually decreased by 19%. They attribute this to inefficient task delegation, though many factors may contribute. Only 44% of code generations were accepted, and 9% of time was spent waiting for AI. Importantly, the agent used had limited capabilities (early 2025), and was used in interactive rather than autonomous mode. Becker et al. [4] started a follow-up study, that tended to show a speed-up instead. However, they did not conclude this study due to strong selection effects: developers were increasingly not willing to work without AI assistance.

Kumar et al. [17] studied the barriers to success of coding agent use with observations from 19 developers using Cursor. The analysis found that developers that delegated the whole task to the agent were less successful than the ones splitting them into subtasks. The main barrier was the lack of project-specific or tacit knowledge of the agent; providing it improved success.

MSR studies of coding agents. Concurrent to this work, a few studies using MSR techniques have emerged. Li et al. present AIDev, a dataset of coding agent pull requests [20], showing a sharp increase in the number of pull requests authored by a subset of agents (Codex, Devin, Copilot, Cursor, and Claude Code). They classify the PRs according to their conventional commit type, with results similar to ours in RQ6.

Mohsenimofidi et al. [22] performed a preliminary qualitative study of the content found in coding agent guidance files, finding that the content is very diverse, and that the most common categories of content include conventions and best practices, contribution guidelines, project structure, and build and testing instructions, among others.

He et al. study projects adopting an early coding agent, Cursor, to find whether it is associated to increases in productivity and quality issues using a difference in differences design [13]. They find an increase in velocity in the first two months of cursor adoption, while static analysis warnings and code complexity increase over time.

Bouzenia and Pradel [6] analyzed autonomous interaction traces from three early academic agents on SWE-bench tasks, revealing significant variability across agents. This variability spans trace length, type, and the differences between successful and failed attempts.

This work. This work applies MSR techniques to study the specific topic of the *adoption* of coding agents. Compared to experiments and observation studies of users of coding agents, our work has a much larger scale (130,000 repositories vs a few dozen participants). Our work is primarily quantitative, whereas observation studies can provide qualitative insights. Finally, the existing studies were conducted in early 2025, whereas we include data from 2025 until 2026, offering more recent insights in a fast-changing field.

Compared to the concurrent work studying coding agents via MSR techniques, our study has a different scope (projects, rather than individual artifacts such as files or PRs). Since we study adoption, we try to be as exhaustive as we can in detecting coding agents (we detect dozens of agents), rather than focusing on a small subset of agents. Another aspect of the exhaustivity is the scale of the study, which is comparable to AIDev, but larger than the other MSR studies. Finally, we study adoption via a variety of lenses, such as adoption by programming language, by type of project as denoted by GitHub topics, adoption in specific organizations, or its evolution over time.

4 Methodology

We study the adoption of coding agents in the real world, at a very large scale, a point of view that is highly complementary to the existing studies, obtained from observations of very few developers [32]. We decided to target open-source GitHub repositories, so the data comes from Git repositories and is available through the GitHub APIs. Figure 2 presents the study at a glance. Section 4.1 details the types of heuristics we use to discover coding agent traces; Section 4.2 describe the data gathering pipeline we employ; Section 4.3 discusses the metrics we gather for the study; finally, Section 4.4 presents the research questions. The code to gather and analyze data from GitHub repositories is available at <https://github.com/labri-progress/agent-impact/>, while the data generated by the pipeline on February 21st, 2026 is available at <https://zenodo.org/records/19256968>.

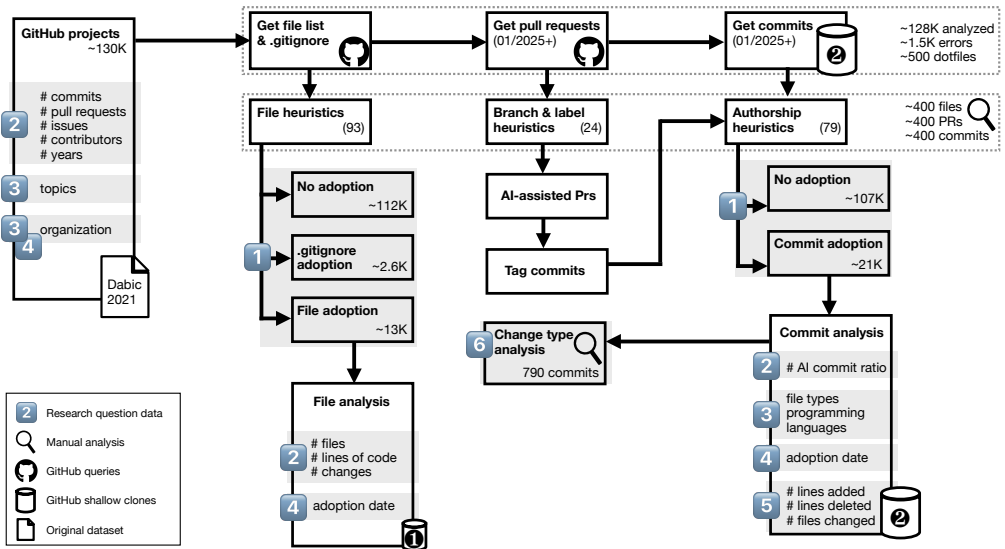


Fig. 2. Overall schema of the study, covering the analysis pipeline, the data sources, heuristics, metrics, and research questions.

4.1 Heuristics for Detecting Coding Agents

One significant challenge is the diversity of the ways the different agents work, as well as the sheer number of available agents [28]. Some agents are integrated in IDEs, while others operate on the command line; some agents are very autonomous, primarily interacting via pull requests.

4.1.1 Inclusion criteria. We started this study in early 2025. At the time, the capabilities of coding agents were more limited, and they were used with close supervision. Our initial inclusion criteria was for tools that leverage LLMs to automate coding, for which we could find traces in repositories.

This includes full-fledged coding agents such as Claude Code, as well as tools that focus on smaller-scale activities, such as fixes automated by tools such as CodeRabbit (as evidenced by commits co-authored by CodeRabbit), whose primary focus is code review.

We note that with time, the increase in capabilities of coding agents means that there is a significant gap in capabilities between full-fledged coding agents, and tools with a more specialized scope. Section 8 investigates adoption of individual tools; the most popular of the specialized scope tools, CodeRabbit, accounts for less than 2% of the commits we detect, indicating that the adoption is driven by full-fledged coding agents.

4.1.2 Heuristic identification. We employ a multi-faceted approach to identify repositories using coding agents such as Claude, Codex, and others. The heuristics are grouped into four categories:

- (1) *Files*: presence of specific configuration files or documentation artifacts that are typical for an agent’s usage (e.g., a `.cursorrules` file).
- (2) *Authors*: authorship or co-authorship of a commit by a known coding agent (e.g. Co-Authored-By: Claude <noreply@anthropic.com>).
- (3) *Branches*: creation of branches whose names follow a convention (e.g. `copilot/fix-...`).
- (4) *Labels*: pull requests labelled with the corresponding agent name (e.g. `codex`).

We found the heuristics via a systematic manual investigation involving the following steps:

- (1) Mapping agent names to repository identifiers.
- (2) Studying documentation to learn how agents are configured and what files they use or generate.
- (3) Manually inspecting repositories likely to employ agents and recording relevant artifacts.
- (4) Performing targeted GitHub searches to confirm hypotheses, such as:

```
Path: CLAUDE.md
Co-authored-by: copilot-swe-agent
is:pr label:codex
```

- (5) Cross-checking results to discover new patterns when a repository uses multiple agents simultaneously.

At the time of writing, we maintain 79 author-based heuristics, 93 file-based heuristics, 20 branch-based heuristics, and 4 label-based heuristics for 63 coding agents. All the heuristics and tools are documented in our replication package. In addition, we also maintain a set of general bot patterns, using authorship heuristics (e.g. `dependabot`); we use this to filter commits and focus on human-authored and AI-assisted commits.

It is important to note that the users can hide the traces of AI-assisted commits by removing the metadata or not adding their files to the repository; these commits, therefore, will be identified as human-authored in our study.

4.1.3 Heuristic validation. Each heuristic was rigorously validated for false positives. A notable example concerns the file `CONVENTIONS.md`. While the Aider website recommends placing agent-related information in this file, it is also commonly used to advertise general project conventions to all developers. Because the risk of misclassifying such files outweighs the potential underestimation of Aider usage, we have excluded `CONVENTIONS.md` from our final heuristic set. Similarly, the `AGENTS.md` file was originally introduced by Codex; however, it is now used by dozens of coding agents, including Copilot, Jules, Cursor, and Aider. Consequently, the presence of this file alone is insufficient to reliably identify the coding agent in use.

To quantify the remaining false positives, we analyzed random samples of commits, files, and pull requests flagged by our heuristics (≈ 400 each). This value exceeds the value needed for a standard confidence level of 95% with a 5% error [33]. For each needed sample, we first sampled one project that had both file-level and commit-level adoption, and then sampled among its flagged commits, files, or pull requests, as needed (if the project did not have pull requests, we resampled). Each artefact was checked by two authors to determine whether it was a true positive (TP) or a false positive (FP); they could use an Unsure annotation for borderline cases, and a 404 annotation to indicate that an artefact no longer existed (e.g. a file was deleted, a repository was taken private). We then measured the initial disagreement, before discussing to adjudicate differences and refine the annotations: notably, we differentiate between legitimate false positives and borderline cases.

For files, there were 28 disagreements out of 400 files (7%). Twelve disagreements were between an annotator finding the file, and the other annotating 404 (several files moved, or the default github branch changed, between when the study was carried out and when we checked the heuristics). Ten disagreements were between TP and Unsure, four between TP and FP, and two between FP and Unsure. After adjudicating disagreements, we had 15 404 files, 370 TP, and 15 borderline cases (4%, excluding deleted files), rather than complete false positives. The borderline cases were 2 empty files, and 13 files that were more concerned with agentic code review rather than coding per se; in 8 of those 15 cases, the repository contained other files confirming the use of agents.

For commits, there were 40 disagreements out of 399 commits (10%). There were 16 disagreements FP—TP; 16 TP—Unsure; and 8 FP—Unsure. An important portion of the disagreements were due to an annotator finding more information than the other; in particular, for some commits the information is found in a pull request, rather than the commit itself. After adjudicating disagreements and refining the labels, we found: 2 404, 370 clear TP, 8 FP (2%, excluding 404), and 19 borderline cases (5%, excluding 404). Fourteen of the borderline cases are commits corresponding to smaller fixes (e.g. code review fixes); the five others correspond to scenarios where the verdict is unclear for the commit itself, but neighbouring commits show clear evidence. Of note, a review of the 8 FP by another annotator found information that would have classified three of them as TP (two with Codex, one with Aider).

For pull requests, there were 16 disagreements out of 399 pull requests (4%). One pull request no longer existed. All disagreements stemmed from one annotator finding evidence that the other did not; they were found to be true positives. After adjudicating disagreements, there was a single false positive out of the 398 remaining pull requests (0.25%).

Overall, our evaluation of heuristics found that very few cases are legitimate false positives (0.5 to 1%), with a higher proportion of borderline cases (3%).

4.2 Analysis Pipeline

Since the adoption of AI agents is very recent and evolving rapidly, recent data is needed. This is why we put a particular focus on automation in this work. We ran the last iteration of our analysis pipeline on February 21st, 2026 and the following two days. Our study period starts on 01/01/2025 and ends on 21/02/2026. Early runs of the analysis included 2024: we found that adoption was extremely limited then; as the complexity of our analysis grew, we decided to focus on the period with the most significant adoption.

We first filter repositories to find relevant active repositories, then we detect agent use and collect statistics. We perform different analyses based on the overall data that we have gathered this way.

4.2.1 Data Selection. We start with a sample using the sampling tool by Dabic et al. [8]. The original dataset contains repositories with at least 10 stars. From this dataset, we select projects that satisfy the following characteristics:

- not forks,
- a minimum of 5 000 lines of code (non-blank, non-comment),
- at least 100 commits,
- active in the last three months (at least one commit),

These criteria ensure that we have projects with a reasonable level of maturity, as we enforce minimum thresholds for activity and code size. In particular, we do not want forks to dominate our sample. In most cases, the bulk of development occurs in the “original” project. In some cases, this may cause us to miss interesting projects—e.g., when an original repository is now inactive but a fork continues development—but we consider this a reasonable compromise.

We accessed the sampling tool to download the dataset used in the remainder of the study on 29/08/2025. After applying the filters above, we are left with a set of 130,621 projects. Considering that GitHub hosts hundreds of millions of projects, we were initially surprised at this relatively low number, given that the sampling tool by Dabic et al. monitors all public projects on GitHub. However, the minimum threshold of 10 stars excludes the vast majority of repositories, and the exclusion of forks substantially reduces the pool of candidates. The minimum activity and size thresholds reduce it further.

We initially considered using Software Heritage [7], as it is the most comprehensive dataset of public source code. However, Software Heritage lags behind GitHub by a few months, which is problematic in our case, as we want data to be as fresh as possible to study such a rapidly changing phenomenon. Beyond the project names and URLs, this dataset includes an important number of metrics (e.g., number of contributors, number of commits, number of forks, etc.), some of which we use in the remainder of this study.

At the end of the analysis process described next, we have data on 128,575 projects, the difference with the original number (130,621) is due to $\approx 1,500$ failures of the pipeline. In the vast majority of cases, failures occur when a repository cannot be found on GitHub (typically because it was made private between the time we downloaded the list of repositories and the time we fetched the file list). We further filter out ≈ 500 “dotfiles” repositories from the analysis as they are not representative of software development; we carry out the analysis on 128,018 projects.

4.2.2 Identification of Agent Adoption via Tools. Next, we use our file-based heuristics to identify agent adopters in our sample. For each repository selected in step 1, we query the GitHub REST API to download the file list of the main branch when we execute the study (on February 21st, 2026). For very large repositories, the API fails because the file list is too large; in that case, we perform a shallow clone of the repository to obtain the file list. For every repository, we store a single text file containing one file path per line.

We then search each file list for names that match our compiled set of file-based heuristics; any repository containing at least one file identified by a heuristic is classified as an adopter at the file level. Nevertheless, we always match all heuristics: by matching all heuristics, we can identify every tool that a repository might use, since a repository may employ more than one agent, and every file that matches heuristics.

4.2.3 Identification of Agent Adoption via .gitignore. Although knowledge in coding agent files is valuable, and thus *should* be shared in the repository, this is not always the case. To have a better coverage of agent adoption, we also look for the presence of agent configuration files in the .gitignore file, which would prevent them from being committed to the repository despite being present. In this way, we identify potential adopters who do not wish to commit certain (or all) agent configuration files. To do so, we query the GitHub API for the main .gitignore file, and we scan it for heuristics in the same way we do for the file list.

4.2.4 Collecting Agent File Statistics. For each of the repositories that we have classified as an adopter, we collect fine-grained information about the agent configuration files that signal agent usage.

- (1) **Shallow clone.** We clone each repository with `-depth 1` and `-no-checkout`, and populate the index with the information of the files in the latest commit, so that the working tree contains only the minimal information to carry out the next steps, to keep storage needs low (even then, the whole analysis still requires more than one terabyte in total).
- (2) **Checkout matched files.** After cloning, we check out all agent configuration files—those that matched our heuristics (e.g., `CLAUDE.md`, `.cursorrules`). This step lets us confirm the correctness of the heuristics by inspecting the actual file contents, if needed.
- (3) **Gather per-file statistics.** For each agent configuration file, we query its commit history, computing the following descriptive statistics:
 - the date on which it was first added,
 - the total number of commits that modified the file (including the initial addition),
 - the size, in lines, of the most recent version.

These metrics provide a quantitative view of how and when agents are introduced into open-source projects, and they serve as the basis for subsequent analyses, such as maturity classification and usage trends.

4.2.5 Commit and Pull Request Level Adoption. After having identified the repositories that contain agent configuration files, we perform a more granular analysis on their commit history and pull requests. In addition, we carry the same analysis on *all* non-adopter repositories to uncover “silent adopters” (repositories that use agents but have no explicit agent configuration file).

Pull Request extraction. For each repository we use the GitHub GraphQL API to download up to 10,000 pull requests (PR) that were created on or after 01/01/2025 and their metadata, including their commits (only 45 projects exceed that number). This includes the associated branch of the pull request, as well as any labels associated with it. These two data sources are the two sources of information for our PR identification heuristics. If a pull request matches one of our branch or PR heuristics, it is tagged as *AI-assisted*. Inside such an *AI-assisted* PR, we tag all the commits by the PR author as *AI-assisted*. This allows us to detect commit activity for agents that do not leave markers in commits, but do so in PRs, such as Codex. For each PR, we also determine its merge status, and merge type (merge, rebase, squash), in order to locate the commits that appear in the main branch.

Commit extraction. For each repository we study, we perform the following steps:

- (1) **Repository clone (historical snapshot).** Given the storage requirements, we perform a *second* shallow clone of each project up, with all the commits from 01/01/2025 to 21/02/2026. As of 01/01/2025, coding agents were still very confidential (as confirmed by an earlier run of the analysis that included 2024): established tools like Copilot and Cursor allowed conversations inside the IDE, but started to propose agent-like functionality in 2025.
- (2) **Commit log parsing.** For scalability reasons, we do not checkout files or compute diffs. From each cloned repository, we run “git log” to obtain: (a) the commit message; (b) authorship information, including any “Co-authored-by” trailers (extracted from the commit message); (c) the list of files (and file paths) affected by the commit, as well as their status; (d) file-change statistics (added/deleted lines) for each file.
- (3) **Commit-level heuristics.** We match each commit against our list of commit-level heuristics (e.g., specific message patterns or authorship attributions). If any heuristic matches, the commit is classified as *AI-assisted*.

- (4) **Linking commits to pull requests.** For each main branch commit, we check whether it appears in at least one AI-assisted PR, and was labelled as AI-assisted there. We take in account the merge type (commit merges with rebase or squash will have distinct commit hashes [5]). If so, we label the commit as AI-assisted. This choice is motivated by the fact that a commit’s provenance is often clearer when it belongs to an AI-assisted PR (such as for Codex). For “squash merged” PR, this adds some imprecision, since the PR is condensed in a single commit which may be only partially AI-assisted. An alternative would be to treat only commits that match our commit-level heuristics as AI-assisted, regardless of PR association. We adopt the former strategy while documenting the trade-off.
- (5) **Bot exclusions.** Commits authored by known non-AI bots (e.g., CI or release bots) are identified via a manually curated list and labelled. They are excluded from the metrics computed in this study, as we wish to get insights on the adoption of AI coding agents in relation to human development activities. The remaining commits are classified as human-authored.
- (6) **Filtering out noise.** Merge commits, revert commits, bump-version releases, and other housekeeping changes are removed to focus on substantive work.

The result of this pipeline is a per-repository catalog of AI-assisted commits, both at the PR level (PR-AI-assisted) and at the individual commit level. These data form the basis for the subsequent metrics detailed next.

4.3 Metrics

For each adopter repository, we use the following metrics. We indicate in which research questions they are used for convenience; the research questions are described next.

Overall adoption metrics (RQ1). We define overall adoption metrics as the proportion of projects exhibiting various indicators of adoption over all projects. We study adoption at the file level (including explicit files and .gitignore files), at the commit level, and overall.

Basic project metrics (RQ2, RQ3, RQ4). From our initial dataset, we extract the following metrics for each project: age in years; number of commits; number of pull requests; number of issues; number of contributors (used in RQ2). We also extract the list of GitHub topics that were assigned to the repository, and to whom it belongs (user or organization); we use these metrics in RQ3 and RQ4.

File adoption statistics (RQ2). We aggregate the file-level adoption data into three metrics: the total number of agent configuration files we detected, the cumulative size (in lines of code) of all such files, and the total number of commits that touched any such files.

Commit-level statistics (RQ5). For every commit that has been labeled as AI-assisted, or human-authored, we extract the following raw numbers from “git log”: the count of lines added and deleted, and the number of files that were added, modified, or removed. These figures provide a granular view of how much work is performed in each commit.

Adoption date (RQ4). We define the *adoption* date as the earliest commit that either adds an agent configuration file to the repository or that is identified as AI-assisted by our heuristics. We define an adoption date for each agent the project uses, and an overall adoption date as the earliest such date.

File-type and language classification (RQ3). Using GitHub Linguist’s file-extension database, we classify each file in a commit as either *source code*, *documentation*, *configuration*, or *other* (data,

images, etc). The same lookup is then used to assign a programming language. Because we only rely on the extension, we avoid the prohibitive cost of parsing every file's contents at scale.

Sometimes an extension maps to multiple languages (e.g., ".rs" could be the Rust programming language, or the deprecated RenderScript language). In important cases, we resolve the ambiguity by choosing the most popular language for that extension in our dataset. The resulting per-commit tables contain, for each file type and programming language, the same line-add/delete statistics as above, enabling downstream analyses such as agent activity by language.

Commit ratio (RQ2, RQ3). We compute an *AI-assisted commit ratio* by selecting all human-authored and AI-assisted commits from a project's adoption date until the end of the study. We then define the commit ratio as number of AI-assisted commits divided by all AI-assisted and Human-authored commits. We compute an overall commit ratio, as well as a commit ratio for each type of files (e.g. source code, documentation).

In addition we use the overall commit ratio to categorize projects in terms of their *commit-level adoption*. For any chosen time window we classify a repository into one of five commit-level adoption levels:

- **No Commits:** The period contains no AI-assisted commits.
- **Experimental:** An AI commit ratio below 1 %.
- **Limited Use** A ratio between 1 % and 5 %, but still below 5 %.
- **Consistent Use** AI commits represent 5–20 % of all activity in the period.
- **Pervasive** More than 20 % of commits are AI-assisted, indicating widespread use.

Note that when analyzing categories of projects in RQ2 and RQ3, we filter out projects with a low number of post-adoption commits (less than 10), to avoid very high or low commit ratios due to limited data.

4.4 Research questions

To carry out our study, we formulate the following research questions:

- **RQ1. What is the estimated adoption of coding agents on GitHub?** We answer this question in Section 5 by analyzing file and commit-level adoption overall.
- **RQ2. What are the characteristics of projects adopting coding agents?** We answer this question in Section 6 by analyzing the distribution of commit adoption metrics, and its relationship with basic project metrics, such as size and age.
- **RQ3. In which context are coding agents used?** We answer this in Section 7 by looking at adoption for specific organizations, topics, and programming languages.
- **RQ4. How has adoption evolved, and which agents are driving it?** We look at the evolution of adoption over time, and at specific coding agents, in Section 8.
- **RQ5. How large are AI-assisted contributions?** We look at the size of commits (co)-authored by agents, and compare it to commits authored by humans and bots in Section 9.
- **RQ6. What types of contributions are AI-assisted?** We classify a sample of commits in various types, following the *conventional commit* classification, in Section 10.

5 RQ1: estimation of overall adoption

Table 1 presents our overall statistics on the datasets of projects we study. We study the adoption along several dimensions: (1) at the file level, on all the projects (with and without the .gitignore file); (2) at the commit level, based on a sample of projects; (3) based on the earlier, we extrapolate for overall adoption rates according to two scenarios.

Table 1. Overall adoption statistics. For each statistic, the middle rows show the subset of the data the adoption is measured in.

Metric	File	Ignored	Commit	Count	Out of	Percent
File-level tool usage						
File-level use	✓	–	–	12,861	128,018	10.05%
Ignored files (all)	–	✓	–	5,020	128,018	3.92%
Ignored files (only)	✗	✓	–	2,606	128,018	2.04%
All file-level use	✓(either)		–	15,467	128,018	12.08%
Commit-level tool usage						
Commit use (file level)	✓	–	✓	8,301	12,861	64.54%
Commit use (all files)	✓(either)		✓	9,440	15,467	61.03%
Commit use, no files	✗	✗	✓	12,951	112,551	11.51%
High Estimate Commit Users	✗	✗	✓	≈ 21,220	112,551	$\frac{11.51\%}{61.03\%}$
Adoption estimates						
Conservative Adoption Estimate	–	–	–	28,418	128,018	22.20%
High Estimate Overall Adoption Rate	–	–	–	≈ 36,687	128,018	28.66%

5.1 File-based adoption

The first section of Table 1 shows adoption of coding agents based on file-level heuristics. Using our file-based heuristics, we find that out of our dataset of 128,018 projects, 12,861 have files indicating that they use coding agents in their repositories (10.05%). In addition, 5,020 projects instruct their version control *not to commit* files corresponding to our heuristics via their `.gitignore` file; of these 2,606 projects have *no visible coding agent files* in their repository, save their mention in the `.gitignore`. Summing both together, we find that 15,467 (12,861 + 2,606) projects reference coding agent files in their repositories, or 12.08%.

5.2 Commit-level adoption

The second section of Table 1 presents commit-level statistics.

Commit adoption only. Out of 112,551 projects that do not have traces of coding agents at the file level (our base population, minus the 15,467 file-level adopters), we find that 12,951, or 11.51%, are commit-level adopters. Surprisingly, this is a similar percentage to the percentage of projects presenting file-level adoption.

Overlap between file-level and commit-level adoption. File-level adoption and commit-level adoption have some overlap, but it is far from complete. As we have just seen, an important proportion of projects have commit-level adoption, but no file-level adoption. The reverse is also true: an important proportion of projects that have file-level adoption would not be detected by commit-level heuristics. Out of visible file-level users, almost two thirds (64.54%) are detected by commit-level heuristics; adding projects that are visible only by their `.gitignore` file, the proportion drops to 61.03%.

5.3 Adoption estimates

The third section of Table 1 extrapolates the adoption rate in the entire population according to two scenarios.

The *conservative scenario* simply sums up the projects that have adoption at the file level (15,467) with the ones that have adoption at the commit level (12,951). This brings us to a total of ≈ 28,418, or an adoption rate of 22.20%.

The *high estimate* scenario is based on the observation that a large proportion of projects (almost half) that are visible at the file levels, are not visible at the commit level. Therefore, it is possible that projects have adopted coding agents, without having any visible files in their repository, nor visible commits. An example of this would be a user of Claude Code that stores their configuration files in a separate repository such as a `dotfiles` repository, and either instructs Claude Code not to sign the commits, or prefers to commit manually (Section 11.4 mentions concrete examples). If we assume that the proportion of such projects is comparable to the ones that have file markers, then only $\approx 61.03\%$ of the projects are detected by the commit heuristics. Following this assumption leads to an estimated number of projects adopting coding agents among those that do not present file-level adoption of $112,551 \times \frac{11.51\%}{61.03\%} = 21,220$. This leads to an estimated percentage of 28.66% in the whole population when we also take into account the file-based users.

Needless to say, given that these projects are very difficult to observe, we can not know their true proportion, making this estimate a very rough approximation. Overall, as of February 21st, 2026, we estimate the true adoption to be somewhere in between our conservative estimate and our high estimate. Regardless of its definitive value, the fact that coding agents, a tool category that saw early products in 2024, and more established products in the Spring of 2025, have already reached 22 to 28% of adoption among GitHub projects barely a year later points to an extremely rapid adoption.

6 RQ2: Adoption and project characteristics

To dive into the characteristics of adoption, we first examine the distribution of file and commit-level adoption. We then analyze how adoption varies with high-level project metrics, such as size, age, or activity.

6.1 Extant of file and commit level adoption

Table 2. Binned distributions of file and commit level adoption metrics

Metric	Bin 1	Bin 2	Bin 3	Bin 4	Bin 5	Bin 6	Bin 7
 Total Files	1	2-5	6-10	11-20	21+		
	52.0%	31.8%	8.0%	4.2%	3.9%		
 Total Lines	0-10	11-50	51-100	101-250	251-500	501-1000	1001+
	5.1%	13.8%	14.4%	29.0%	16.0%	9.3%	12.4%
 Total Changes	1	2-5	6-10	11-20	21+		
	21.8%	36.4%	16.3%	11.8%	13.8%		
 Adoption Ratio	None	Experimental	Limited	Consistent	Pervasive		
	34.6%	7.4%	18.4%	20.7%	18.8%		

Table 2 shows statistics on the distribution of file-level adoption in terms of number of files, lines in these files, and number of changes, and of commit-level adoption. This analysis excludes projects which we detect solely via `.gitignore`, as we cannot measure these metrics for them. We grouped the metrics in bins to ease the interpretation of the data by providing reference points.

Number of files. We see that more than half of projects (52.0%) have a single rule or guidance file, and more than 16% of projects have 6 files or more. Further, a small amount of projects have an extensive amount of guidance files (more than 10: 4.2%; more than 20: 3.9%), indicating that some projects use either sophisticated guidance for their coding agents, or multiple agents.

Number of lines. In terms of the amount of content in the rules and guidance files, we see that a small number of projects have very short guidance (10 lines or less: 5.1%). Such a small amount indicates projects having default or very basic configurations. The bulk of the projects have between 101 and 250 lines (29.0%). At the extreme end, more than 10% of projects have more than 1,000 lines of guidance (12.4%), once again indicating sophisticated guidance.

Number of changes. In terms of changes, we see that almost a quarter of projects added their guidance file and never changed it (21.8%), while most projects occasionally change or add new guidance files (36.4%). On the other hand, some projects update their guidance regularly (more than 10 additions or changes: 11.8%; more than 20: 13.8%).

Commit ratio. In terms of commit ratio (shown on the last row), we see that, as shown in Table 1, 34.6% of file-using projects have no commit-level activity. Of the remainder, we see that a majority of the projects have significant of commit level adoption: *Consistent* users (5 to 20% of AI-assisted commits) are at 20.7%; *Pervasive* users are at a comparable 18.8%. Of note, some of these ratios may be high for projects having low activity (e.g., a project with a single post-adoption commit, that happens to be AI-assisted, will have a commit ratio of 100%). For this reason, in the remainder of the analysis, we only consider projects with at least 10 post-adoption commits.

Relationship between the metrics. Figure 3 shows the distribution of the four adoption metrics, and how each pair of metrics relate to each other via scatterplots and hexbin plots (showing density); we also compute the correlation. Since many projects have an adoption ratio of 0, we focus only on the ones with a positive commit ratio; further, we include only projects with at least 10 post-adoption commits. The figure shows that there is a high degree of correlation between the file-level adoption metrics: projects that have larger or more numerous files also tend to change them more often. The correlation is strongest between number of files and total content, and somewhat weaker between lines of code and changes. On the other hand, even when filtering for absence of commit ratio and low activity, the correlation between the commit ratio and the file-level activity, if positive, is very low (at best, 0.1 with changes). Clearly, there are more factors at play to explain the relationship between the two categories of metrics.

Overall, this points to a large proportion of projects that adopt coding agents, but with a modest amount of guidance. A sizeable minority uses a large amount of actively-maintained guidance for coding agents. Whether and how this translates to a large amount of *visible* coding agent activity is another matter: there is a large minority of projects that have a high visible agent usage; however they are not necessarily the ones with the largest amount of guidance.

6.2 Project characteristics and file-level adoption

We investigate the characteristics of the projects adopting agents. We first focus on the adoption rate at the file level. As earlier, we count projects that show file-level adoption by usages of `.gitignore` as adopters. To look into project-level metrics, we split each metric of interest in our dataset into deciles, and we compute the file-level adoption ratio for each decile. This allows us to see whether projects that have larger/smaller metrics tend to have a larger/smaller adoption ratio.

Table 3 presents the deciles and adoption ratio for 6 metrics: size in (non-blank, non-comment) lines of code, age in years, number of contributors, commits, issues, and pull requests. For better readability, we also depict the trends using Sparklines [35]—word-size graphical representation of trends or distributions of data. While there is a common conception that coding agents work best with “greenfield” projects [26], as it is much easier to write “new” code without needing to integrate it in a large codebase, our results are more nuanced. We describe each metric in turn.

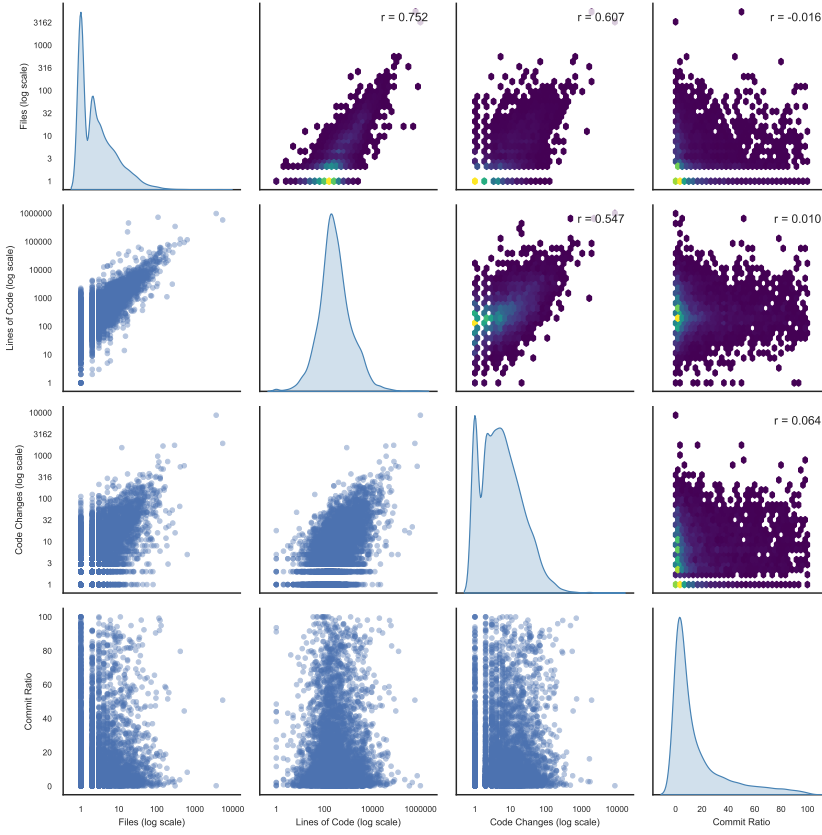


Fig. 3. Pairwise relations of file and commit level adoption metrics

- **Lines of Code:** we first check whether larger projects have more or less adoption than smaller projects. We find that, somewhat contrary to expectations, adoption is *higher* for larger projects. Projects in the smallest decile have an adoption rate of 6.94%, while the largest project categories (from 44K lines of code and up) more than double the adoption rate (15.22%–15.91%). This trend maintains itself for the very largest decile (322K lines of code and up).
- **Contributors:** in terms of contributors, adoption grows smoothly between 9.71% and 13.07%, but grows sharply for the largest decile (68 and up, 17.88%). Note that the contributor count (extracted from GitHub) includes all contributors (human, bots, agents). However, this does not impact the trend that the very largest projects have more adoption.
- **Commits:** for commits, adoption hovers at around 10% for the smallest deciles, before growing smoothly for the largest half of projects, reaching 19.03% for the largest decile (3.9K commits and up).
- **Issues:** as with other metrics, we find that projects with more issues tend to have a higher adoption rate. This is especially the case of the top decile (459+ issues), where the adoption rate is 20.45%.
- **Pull Requests:** the tendency that projects in larger deciles have higher adoption is further confirmed by pull requests, where the difference is even more stark, with the highest decile

Table 3. File adoption statistics versus project-level metrics, by deciles

Metric	Aspect	Sparkline	D1	D2	D3	D4	D5	D6	D7	D8	D9	D10
LOC	Deciles		5.0K	7.5K	11K	15K	21K	30K	44K	71K	129K	322K
	File		6.94%	8.31%	9.29%	9.87%	11.73%	13.06%	14.41%	15.22%	16.07%	15.91%
Contributors	Deciles		0.0	1.0	3.0	4.0	6.0	10	14	21	34	68
	File		9.71%	11.50%	11.63%	10.98%	10.76%	11.53%	11.68%	12.42%	13.07%	17.88%
Commits	Deciles		100	159	233	324	446	610	849	1.2K	1.9K	3.9K
	File		10.07%	9.95%	9.84%	10.15%	10.63%	11.04%	11.49%	13.13%	15.50%	19.03%
Issues	Deciles		0.0	3.0	9.0	19	35	60	105	195	459	
	File		9.63%	10.39%	10.94%	10.31%	10.90%	11.70%	12.75%	14.20%	20.45%	
Pull Request	Deciles		0.0	1.0	8.0	22	46	85	145	244	435	932
	File		6.57%	7.83%	8.60%	10.40%	10.65%	11.38%	11.92%	13.11%	15.29%	25.15%
Age (years)	Deciles		0.0	1.1	2.0	2.9	3.8	4.8	5.8	7.1	8.7	11
	File		26.37%	14.44%	13.00%	10.78%	10.34%	10.21%	9.93%	8.83%	8.53%	7.96%

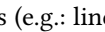
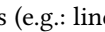
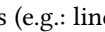
adoption jumping from 15.29% to 25.15%. Note that, depending on their workflows, agents can, and do, contribute pull requests to projects. Thus, it is likely that the stark difference observed is at least partly self-fulfilling, with projects having a heavy usage of coding agents seeing a fast growth in pull requests.

- **Age:** While larger metric values are associated with higher usage for the other metrics, age goes strongly in the other direction. Younger projects have a far higher adoption rate than older projects, with a factor of 3 difference between the smallest decile (1 year or less, 26.37%), and the largest (more than a decade, 7.96%). On its own, this does comfort the narrative that coding agents are best used in greenfield projects; however, we see that even old projects see a non-negligible adoption rate (almost 8%).

Thus, we see two trends at play here: clearly, younger projects have a much higher adoption than older projects. On the other hand, there is also a trend towards larger projects with more contributors, commits, issues, and pull requests having a higher adoption rate. This trend is partially self-fulfilling, since agents are contributors, and can contribute commits and pull requests to a project.

We applied a Chi-square goodness-of-fit test to assess whether the observed values are evenly distributed across categories or tend to be concentrated; in our case, the categories correspond to deciles. We also computed the effect size associated with the Chi-square test. The results show statistically significant differences for all metrics (*p-value* < 0.01), indicating that the distributions are not uniform across bins. The effect sizes are *small* for lines of code, contributors, commits, and issues, and *medium* for pull requests and age.

To tease apart these two effects, we re-ran the same analysis, excluding projects younger than one year. While the values of all the deciles and their adoption ratios changed slightly, we see the same trends, with higher adoption for higher metric values, and decrease of adoption with age (e.g., lines of code: ; contributors: ; commits: ). The main changes were that, as expected, the very high peak for the first decile in age shrank ().

We also carried out the same analysis, excluding projects younger than two years, finding again similar results (e.g.: lines of code: ; issues: ; pull requests: ). These

two analyses comfort the observation that there are two distinct effects: a large effect of age for younger projects; however, even when removing young projects, the trend towards higher adoption of coding agents in projects with higher numbers of lines of code, contributors, commits, issues, or pull requests is still clearly noticeable.

6.3 Project characteristics and commit-level adoption

Table 4 presents our analysis of the AI-assisted commit ratio of the projects in our dataset, since their adoption of coding agents. For this analysis, we only consider projects that have at least 10 commits since adoption; this avoids cases of projects having a very high commit ratio but little activity (e.g., a project that has a 100% AI-assisted commit ratio, but a single commit). We carry out this analysis for several categories of projects, and the per-decile analysis we applied for the file-level usage. We use the commit ratio categories defined in Section 4 (Experimental: < 1%; Limited: < 5%; Consistent: < 20%; Pervasive: >= 20%).

Table 4. Commit adoption across different project categories

Category	N	None	Experimental	Limited	Consistent	Pervasive
All Files	12,027	26.73%	9.16%	22.55%	24.00%	17.56%
by LOC						
by Contributors						
by Age						
All Files with commits	8,812	0.00%	12.51%	30.78%	32.75%	23.97%
by LOC						
by Contributors						
by Age						
File level	7,802	0.00%	12.13%	30.17%	32.71%	24.99%
by LOC						
by Contributors						
by Age						
Files ignored only	1,010	0.00%	15.45%	35.45%	33.07%	16.04%
by LOC						
by Contributors						
by Age						
Commits only	9,640	0.00%	13.49%	36.95%	34.62%	14.95%
by LOC						
by Contributors						
by Age						

Caveats. Note that this analysis comes with important caveats. In particular, there are good reasons to think the AI-assisted commit ratio is an *under estimate* of the actual one, since: 1) not all agents sign their commits or tag their pull requests; 2) even if they do, this behaviour can often be turned off; and 3) some developers may use agents, but prefer to commit manually. This can cause projects to have a commit ratio of 0 if all the activity is unobserved, or just to reduce the commit ratio if, e.g. a developer disables commit signing at a given moment. We expand on this in Section 11.

Overall adoption. The first section of Table 4 shows the distribution of the commit ratio for all the projects that have file adoption markers. Overall, 26.73% of projects have a ratio of AI-assisted commit of 0 (this is smaller than the value among all adopters, $\approx 46\%$, as we filter projects with few commits for this analysis). Out of the projects that have AI-assisted commits, the smallest category

is the “Experimental” category (9.16%). The second smallest is the “Pervasive” category, the one with the largest degree of AI assistance (17.56%). The bulk of the remaining projects are roughly equally distributed between the “Limited” and “Consistent” category.

Since we can not know for certain whether the projects with a ratio of 0% have absolutely no AI-assisted commits or not, we filter them out in the subsequent analysis. Of the remaining projects, we see that a significant proportion of projects have a very high usage of coding agents, with almost a quarter (23.97%) of such projects having more than one in five commits that are AI-assisted. We find this number to be very high, especially considering that it is likely to be an undercount. We note that some projects have very high ratios of AI assistance at the commit level. In particular, 5.25% of projects with non-zero commit adoption have the *majority* of their commits that are AI-assisted; 3.09% have three quarters or more of their commits that are AI-assisted.

As in the prior section, we also applied a Chi-square goodness-of-fit test to assess whether the observed values are evenly distributed across the commit ratio categories. We also computed the effect size associated with the Chi-square test. The results show statistically significant differences for all metrics ($p\text{-value} < 0.01$), confirming that the distributions are not uniform across categories. The effect sizes are *medium* for all files, and *large* for the other metrics.

Adoption across categories. The bottom three sections of Table 4 focus on specific categories of projects, and can be directly compared with the second section, since they all filter projects with no AI-assisted commits (the projects detected by the commit-level heuristics have this by definition). Notably, we see that projects that do not have visible files in their repositories (either detected through `.gitignore` or through commit-level heuristics only) tend to have lower levels of adoption than the ones detected by file-level heuristics. In particular, they have approximately two thirds as much projects in the “Pervasive” category, while having more projects in the remaining categories. In both cases, the relative growth is larger in the “Experimental” and “Limited” categories (≈ 10 to 25%), while it is considerably smaller for the “Consistent” categories (≈ 2 to 5%).

Variation with project metrics. Looking at the variation of the distribution of commit ratio categories across project metrics in deciles paints a contrasting picture with the file-level adoption. We focus on lines of code and contributors, but we see similar trends in the other metrics (commits, issues, and pull requests). In all cases, larger or more established projects (e.g. with more contributors) tend to have a *smaller* ratio of AI-assisted commits: across deciles, the proportion of “Experimental” projects rises steadily, while the proportion of “Pervasive” projects decreases. The pattern is less clear for the “Limited” and “Consistent” categories: for file-level adopters, the “Limited” category rises with deciles, and the “Consistent” category is stable; for projects detected via `.gitignore` or commits, the “Limited” category seems to rise with deciles, while the “Consistent” category seems to decrease, although the trends are noisier due to the smaller size of the sample.

For Age, the trends are less clear: the first decile tends to have a higher proportion of “Pervasive” projects (except for commit-only projects). Beyond the first decile, the trend appears to be relatively stable across all the categories.

If we compare to the file-level adoption, we see that if larger or more established projects have more file-level adoption, this does not translate to a higher level of commit adoption. This echoes the earlier finding that the correlation of the metrics is very low, but gives us additional insights. Adopters that are larger or more established seem to use coding agents in more limited settings than smaller projects. In terms of age, we still see a trend that younger projects are more extensive adopters, similar to the file-level adoption. Beyond that, the age of a project in itself does not appear to be correlated to the intensity of use of coding agents. While larger, more established projects have less adoption at the commit level, it is worth noting that a small but significant proportion of

these projects have high levels of adoption: for instance, the proportion of “Pervasive” file-level adopters in the largest LOC decile is more than 5%.

A final remark is that the proportion of projects that have *no AI-assisted commits* is decreasing with deciles. The effect is particularly visible for the number of contributors. Given that each contributor can have their own setting and workflow for a coding agent, it is possible that projects with more contributors would have a higher probability that at least some of them leave visible traces, leading to this observed behavior.

7 RQ3: contexts of use of coding agents

We refine our analysis of adoption by looking at specific contexts: we start with the adoption in specific organizations, before looking at projects that have specific GitHub topics, before analysing adoption by programming languages.

7.1 Adoption in specific organizations

Table 5. File-based adoption for top 20 organizations

Organization	Adoption pct	Repositories	Organization	Adoption pct	Repositories
Microsoft	25.80%	938	Mozilla	10.96%	73
Google	9.86%	629	Automattic	35.38%	65
Apache	8.99%	534	Shopify	14.52%	62
Amazon	12.74%	416	Alibaba	22.81%	57
Hashicorp	2.27%	132	Rust	3.85%	52
Meta	14.81%	108	Tencent	7.84%	51
Nvidia	15.89%	107	Kubernetes	4.00%	50
Grafana	24.47%	94	Huggingface	16.33%	49
Jetbrains	17.24%	87	Symfony	4.08%	49
Elastic	13.41%	82	Iobroker	100.00%	48

Since our dataset contains projects from large organizations, we were interested in seeing the variation in adoption across these. This is especially relevant since projects belonging to large organizations are generally more mature than other GitHub projects. To do so, we started with a list of the top 100 organizations according to gitstar-ranking¹⁴. We then manually grouped some organizations into groups representing companies (e.g., Microsoft, GitHub, Azure, DotNet are all parts of Microsoft). We then simply compute the file-level adoption ratio for the top twenty of these organizations (for each one, and overall). Over all the repositories in the top twenty organizations, file-level adoption is at 17.21%. For reference, the file-level adoption over all the repositories is 12.08%. This means that among open-source repositories of top organizations, there is a higher adoption of coding agents (a relative increase of $\approx 42\%$).

Looking at specific organizations, Table 5 shows the top 20 organizations and their file-level adoption rate. We can see that there are large variations, with some organizations well below the average. HashiCorp is the lowest (2.27%), followed by Rust, Kubernetes, and Symphony (3.85%, 4.00%, and 4.08%, respectively). Other organizations, on the other hand, are well above the average, notably Microsoft, which, with a few other organizations, has close to double the mean adoption (25.80%); given that Microsoft, through GitHub Copilot, is itself a provider of a leading coding agent, there is certainly a strong interest in using coding agents in practice. Iobroker is of particular interest, having transitioned all of their projects to use agents; the few repositories we checked adopted Copilot.

¹⁴<https://gitstar-ranking.com/organizations>

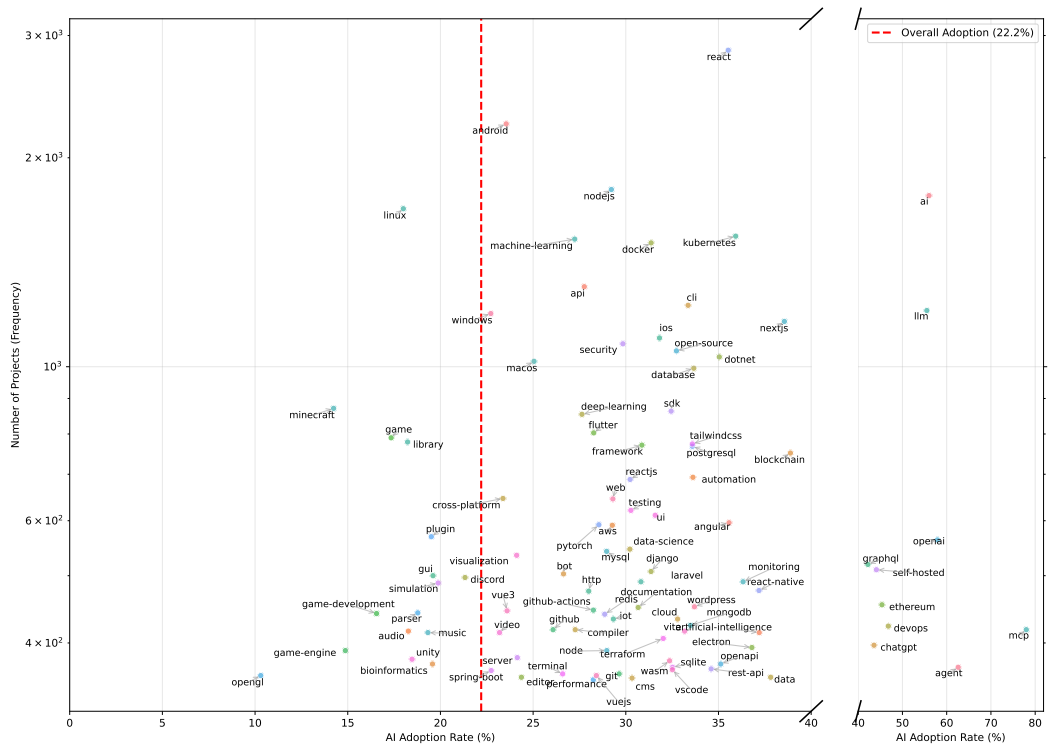


Fig. 4. Topics of GitHub projects showing evidence of the adoption of coding agents, for topics associated to at least 350 projects. Each topic is plotted with adoption rate, i.e., ratio of projects using agents among projects with the same topic, on the X axis; and the number of projects using agents on the Y axis (log scale).

7.2 Adoption by topic

7.2.1 Overall adoption by topic. Figure 4 shows the GitHub topics of projects with evidence of adoption of coding agents. The figure focuses on the top topics in our dataset, i.e., it only shows topics associated to ≥ 350 projects; this threshold was chosen to have both a large enough number of project and ensuring the figure is legible. Note that projects can, and often are, associated to multiple topics; hence a single project can contribute to multiple points in this visualization. For each retained topic we show: the adoption rate on the X axis, as a percentage of projects for that topic that use agents; and the number (frequency) of adopting projects on the Y axis, on a log scale. We filter out programming languages (which we see next), and split the figure in two sections to increase legibility.

The average adoption rate in this dataset is around 22%, and very scattered around topics (X axis). Adoption is below average, for topics like minecraft, system and network programming topics (e.g., those related to game engines, opengl, linux, or network protocols like http). Even topics related to AI but on the more foundational side, like machine-learning and deep-learning are near the average. While some topics are below average, we stress that very few topics have extremely low adoption. To check whether topics below the 350 threshold could have lower adoption, we generated alternative versions of the figure with lower thresholds (down to 100, but far less readable). We found a handful of topics with between 7% and 10% of adoption; it is possible that more niche topics would have lower adoption.

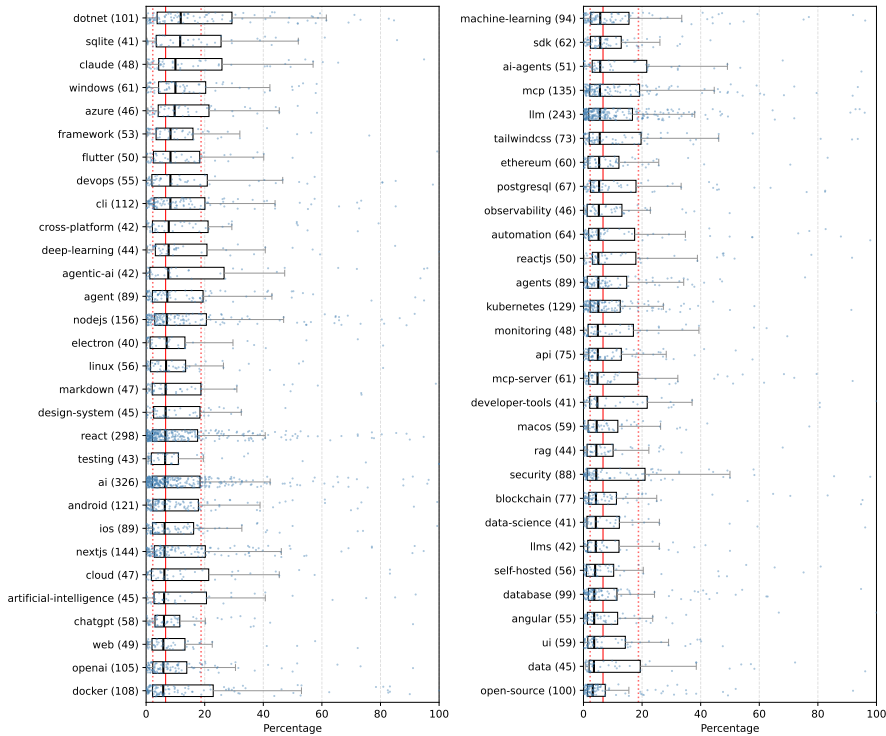


Fig. 5. Commit Ratio distribution by topic of commit-level adopters projects. The red lines shows the overall median commit ratio, while the dotted red lines show quartiles.

In between the average and around 35% (which is still far lower than the upper-end of the scale, above 70%), we find several topics related to Web development (e.g., React, rest-api, wasm, electron), others related to cloud and automation (cloud, docker, terraform, devops), as well as many specific database management systems. We also see a few cross-cutting themes, and technologies, like open-source, dotnet, and vscode (the latter presumably related to coding agents integrated with the popular IDE).

Among the topics with highest adoption ratio (35–80%) some entries can be interpreted as “dog fooding”, i.e., projects that either develop agents themselves (and use agents for agent development!), e.g.: agent. Other high-adoption topics are for neighboring technologies, like mcp, but also ai, llm, chatgpt, and openai. Other high-adopter topics are more surprising, like angular, nextjs, and ethereum; the communities behind blockchains and some frontend Web development frameworks appear to be more likely to use coding agents than other technology communities. Investigating why this is the case is beyond the scope of this paper, and left as interesting future work to pursue.

7.2.2 Commit-level adoption by topic. Figure 5 shows the distribution of commit ratios among commit-level adopters categorized by topic. A boxplot shows the distribution for a given topic, while a stripplot gives insights on the distribution, such as outliers. The figure also shows the median commit ratio over all commit adopters (6.7%) as a red line, and the quartiles (2.3% and 18.6%) as red dotted lines. As earlier, we only consider projects with at least 10 post-adoption commits.

Unlike Figure 4 which shows a high discrepancy of file-level adoption depending on the topic, Figure 5 shows a more homogeneous commit ratio across topics. This observation confirms the

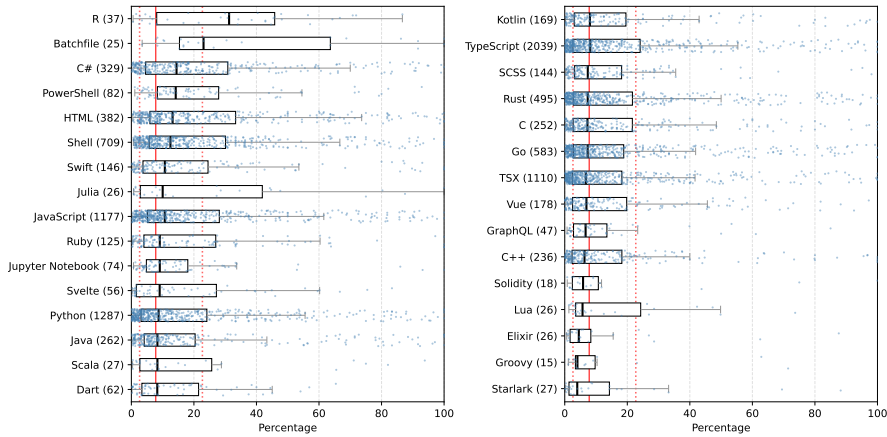


Fig. 6. Commit Ratio distribution by programming language of commit-level adopters projects

data from Figure 3, where we saw that the correlation between file-level and commit-level activity is very low: similarly, there appears to be little correlation between adoption rate based on topics, and commit ratio based on topics. We would have thought that topics linked to AI, LLMs or agents had a higher commit ratio than other topics such as android or linux, but that is not the case. For instance, the topics `openai` and `llm` have a very high file-level adoption rate whereas their median commit ratio is at or slightly below the average median commit ratio among commit-level adopters. One possible reason for that is that developers for these topics have a higher awareness of the workings of coding agents, and are more likely to change their configuration, leading to a decrease in observable traces.

The leaders in commit ratio are `dotnet`, `sqlite`, and `claude` which have a median commit ratio a few percentage higher than the overall median. Some topics have somewhat lower commit ratios distributions, notably `ui`, `data`, and `open-source`.

7.2.3 Takeaway. The main takeaway of this analysis is that adoption is *broad*: at the file level, it spans a diversity of topics, with topics with very high adoption, but surprisingly few topics with very limited adoption. At the commit level, adoption is more uniform, showing again the diversity of topics in which coding agents are used.

7.3 Adoption by programming language

For programming languages we only show the commit-level data analysis, as it is much more precise. When computing commit ratio, we count commits that affect files belonging to a given programming language (identified by GitHub linguist). We only consider projects with at least 10 post-adoption commits, but apply this filter for the language of interest.

Figure 6 shows the distribution of commit ratios among commit-level adopters categorized by programming language. For each language, a boxplot shows the distribution of commit ratios, while a stripplot gives more insights such as on the outliers. The figure also shows the median commit ratio over all commit adopters (7.8%) as a red line, and the quartiles (2.6% and 22.7%) as red dotted lines. Interestingly, the commit ratios on source code are higher than the overall commit ratios, showing that, indeed, coding agents are used for coding, but also for other activities (such as documentation).

The data shows some differences but overall most single programming languages see comparable commit ratios within $\pm 10\%$. Surprisingly, languages such as C, C++ or Rust who were expected to have lower commit ratios, due to their low-level nature, are in the middle of the pack, with close to the same median as all programming languages overall. Widely used languages such as Javascript, Python, Java have also median commit ratios close to the overall median.

Interestingly, some less popular languages such as Dart, Swift, have commit ratios comparable to the most popular languages, while Kotlin and Julia even have relatively higher commit ratios. This shows that coding agents have some usefulness beyond the very popular languages with ample training data. However, less established languages such as GraphQL, SCSS, Svelte and Elixir are trailing in median commit ratio compared to the others. Doing this analysis for additional languages would be interesting, but our filters reduce the number of repositories, preventing this analysis.

Some languages are positive outliers, notably R and Batch, although the relatively low number of projects prevents us to make strong conclusions. Among more established languages, the leaders are C#, Shell, HTML, and PowerShell which have medians up to nearly 8% above the overall median. Shell languages are good use cases for coding agents: they typically are small-size scripts, in languages that are common, yet are used less frequently day-to-day by developers.

Notice that there is a high variance in commit ratio for most programming languages. Most commit-level adopters have commit ratios under the 20% but there are always some projects with very high commit ratio from 40% up to 100%. This is reflected in the upper quartile, which tends to be higher than in Figure 5.

All in all, this analysis also comforts the idea that adoption is broad: it is not limited to a small subset of very popular languages, but extends also to less popular ones.

8 RQ4: evolution of adoption over time, and tool-specific adoption

In this section, we investigate the evolution of adoption over time, as well as the adoption of specific tools.

8.1 Evolution of adoption over time

Figure 7 presents the cumulative adoption of coding accross all the projects for which we could identify it. We record as adoption date the earliest date for which we have evidence of adoption, either through files or through commits. We can see several inflections point on the curve:

- At the end of February 2025, the slope increases, indicating a faster take-off; this corresponds to the initial release of Claude Code.
- The slope increases further around mid-May 2025; from then on the adoption occurred at the fastest rate. Around this period, multiple coding agents were released (e.g. Codex, Gemini, Jules), which likely raised the popularity of the whole category.
- Adoption seems to slow down in the fall of 2025, from September to December, but the slope is still higher than it was before the second inflection point.
- Finally, there is a fourth inflection point at the start of 2026, where adoption picks up sharply once again.

Overall, we see a fast increase of adoption. An earlier version of this study detected hints of a possible slow down in the fall of 2025, but, as of early 2026, the trend has picked up again. This is consistent with the reported increase in capabilities of the models released at the end of 2025 (e.g. Claude Opus 4.5), with training particularly focusing on improving agentic tasks and tool use. Improvement in agentic capabilities has continued, with additional model releases in February and March 2026; however it is too early to foresee their impact on adoption.

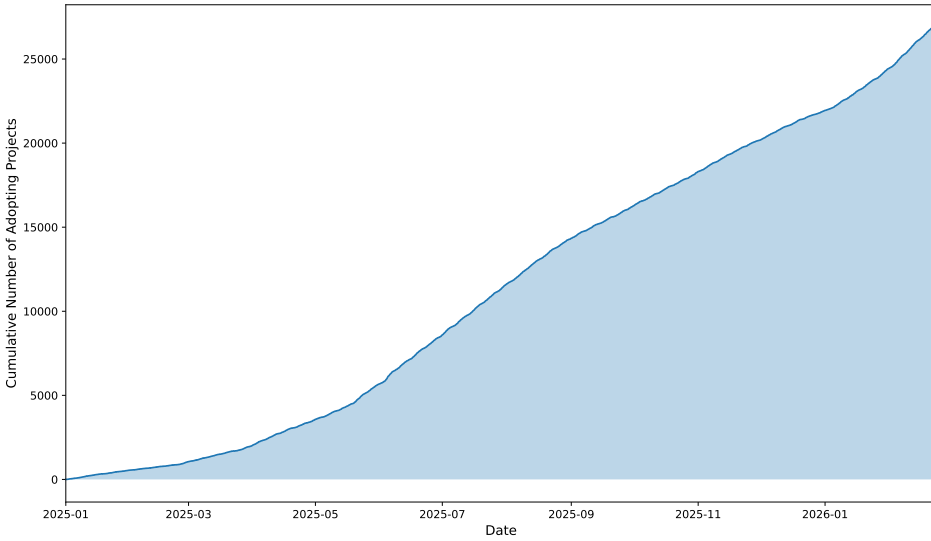


Fig. 7. Cumulative adoption of any tool across all 26 883 projects from 2025-01-01 to 2026-02-23.

8.2 Adoption per tool

Figure 8 shows the evolution of the adoption for the 54 specific tools that we track, via a two-panel ridgeline plot. Overall, the 26 883 projects have adopted 42 060 tools; thus some projects adopt multiple tools (we analyse this next).

We separate the visualization in two parts as there are wide differences in terms of popularity across tools. The left part shows the 6 most popular tools (all with more than 500 adopting projects), as well as an “other” category for the remaining tools. The right panel zooms in on this “other” category, specifically on the next 15 most popular tools (50 or more adopting projects). The remaining 34 tools that we identified are shown in the rightmost “other” category; these tools have a more limited adoption, but make a sizeable minority together.

Of note, the “Generic” category is made up of projects that use the `AGENTS.md` guidance file. This was initially the file used for the Codex coding agent; however, a standardization effort has started on this file format as the default guidance file for coding agents¹⁵. This is why we can no longer attribute it with certainty to Codex, although we suspect that the majority of its usage stems from Codex.

Figure 8 shows that a minority of tools show a significantly higher adoption than the rest. The top 5 tools (assuming most of Generic tool usage is Codex) account for more than 80% of the overall adoption. Just Claude and Copilot are responsible for more than half of the adoption.

If we remove the 132 projects that adopt both (see Figure 9), and estimate that $\approx 90\%$ of the remainder is likely projects adopting Codex ($\approx 3,850$) rather than other agents, we arrive at an estimate of $\approx 6,000$ projects adopting Codex, which would place it in a clear third position behind Claude and Copilot. We stress this is a very rough estimate.

In terms of trends, we clearly see all the inflection points on the Claude trendline. On the other hand, the trendlines of Copilot and Cursor are more regular, although Cursor picks up at the

¹⁵<https://agents.md>

last inflection point. The trend for Codex, if we also account for the “Generic” trend, also shows a significant increase starting in the second half of 2025, with a sharper turn upwards in 2026.

In the remainder of projects, we see different trends: some projects such as Aider, Devin, or Coderabbit have been available for longer (starting in 2024), and seem to rise steadily over time. Others (such as Amp, OpenCode, Kiro or Serena) are more recent agents, who may increase in popularity in the future.

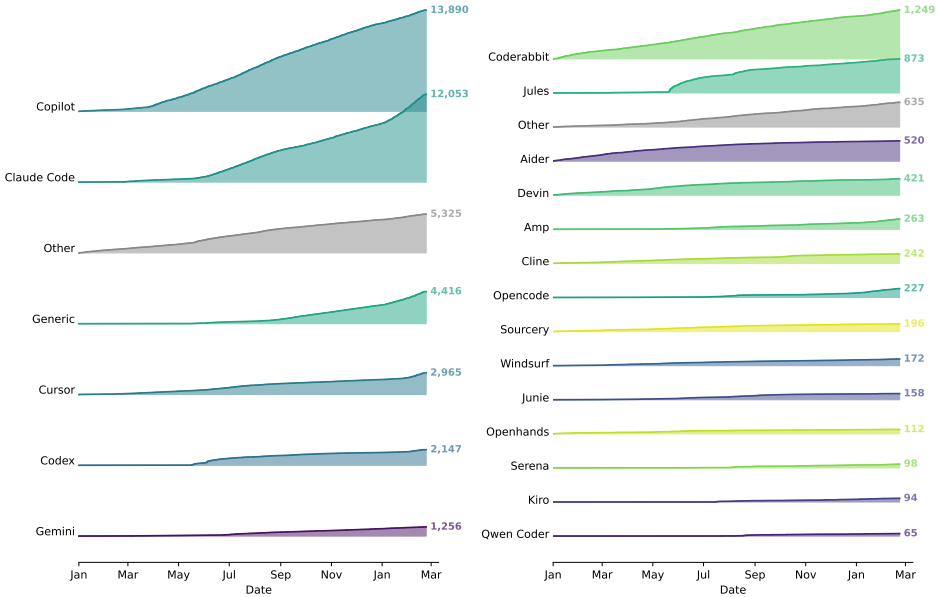


Fig. 8. Evolution of the number of projects (Y axis) that adopt each tool over time for the 54 tools, with Copilot being the most adopted (13 890 projects).

8.3 Co-adoption of coding agents

As mentioned earlier, the 26 883 projects have adopted 42 060 tools, representing more than 1.6 adopted agents per adopting project, which is why we analyse the projects that have adopted more than one coding agent. Overall, two thirds of projects use a single tool (17,774 projects); the remaining third uses multiple tools, confirming significant co-adoption. More precisely, 5,570 projects use two tools, 2,072 use three, and 1,467 use four tools or more, with outliers using ten or more.

Figure 9 presents an UpSet plot of the co-occurrences. An UpSet plot is similar to a Venn diagram, but can scale more gracefully as the number of categories increase [19]. The figure displays the number of projects that use multiple agents, filtering for cases where at least 40 projects adopt more than one combination of agents, limiting to agents with at least 250 adopters for legibility. Each agent is represented by a horizontal line. Each combination (ranging from one to five agents in the figure) is represented as a series of points, connected by a line if there are more than one coding agents in the combination. Simpler combination and more frequent combinations are on the left, so the figure starts with single usage of the most popular agents, and ends with the complex combinations. The plot also includes horizontal and vertical bar charts to show: 1) the count of

projects using a particular agent vertically, and 2) the count of projects using a particular agent combination horizontally. The figure gives insights on common agent combinations; e.g., the most popular is Claude_Code $\cap$ Copilot (1732).

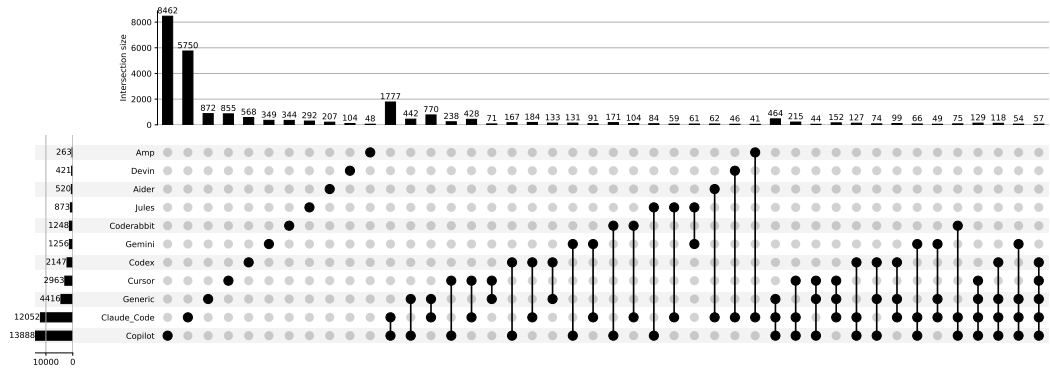


Fig. 9. UpSet plot showing tool co-adoption patterns among 11 tools with at least 40 projects per intersection.

8.4 Adoption evolution in organizations

Figure 10 shows the evolution of adoption across the larger organization in our dataset over time. Note that the list is different from the top 20 organizations we had earlier, as this one focuses on the organizations with the most adoption. We once again use a two-sided ridgeline plot.

We see that at the organization level, the adoption has a few distinct profiles. In particular some organizations, once they start to adopt coding agents, do so very quickly (e.g., Micronaut), and in some cases, almost overnight (e.g., iobroker), or by waves (Pulumi). Other organizations have a more regular adoption (e.g. Apache, Google).

We also see similar inflection points than in the overall adoption curve (Figure 7). Notably, some organization start to adopt coding agents in February or March (e.g. Automatic, Antiwork, Posthog), and there is a significant takeoff in the May to July period (e.g., Getsentry, Datadog, Reown). Finally, there is clearly a significant inflection point in late 2025/early 2026, with several organizations seeing a rise at that time (e.g., Nvidia, Elastic, Mongodb, among others).

Very few top organizations had adoption as soon as January 2025: Microsoft is the exception, being the earliest and the largest overall adopter. Given that they are a provider of coding agents with Copilot, Microsoft certainly has some incentives to adopt it itself and serve as an example.

9 RQ5: Size of AI contributions

Beyond the commit ratio analysis, in this section, we compare the size of AI-assisted contributions with the size of human-written contributions. To do so, we measure the size of commits with three metrics: 1), the number of lines added in the commit; 2), the number of lines deleted in the commit; and 3), the number of files involved in the commit (added, deleted, modified, or renamed). We limit the number of metrics to simplify the presentation. We then use our heuristics to determine if a commit was authored by a human, a software bot, or was assisted by a coding agent. We do this analysis for all commits by file-level adopters that have commit-level adoption. This allows us to minimize the number of AI-assisted commits mislabelled as human contributions, although this mitigation is not perfect (e.g., a contributor may decide to stop their coding agent signing commits, while still using it).

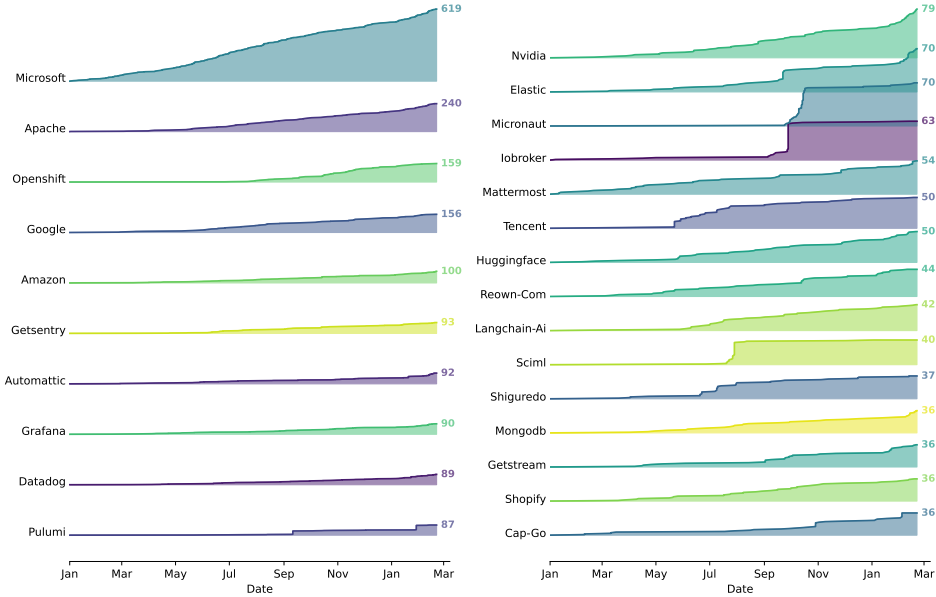


Fig. 10. Evolution of the number of projects per organization (Y axis) that adopt coding agents over time across the top 35 organizations, with microsoft leading at 619 adoptions.

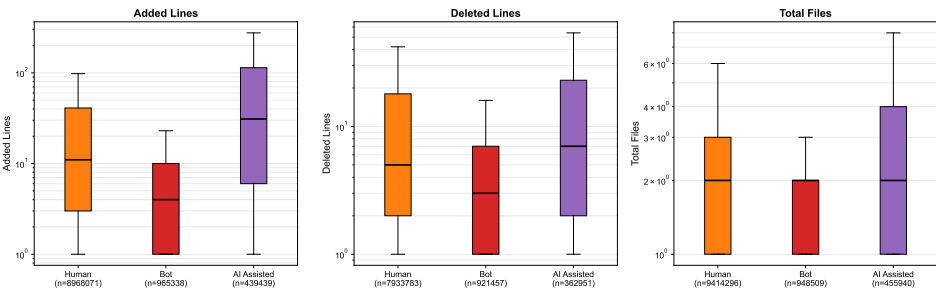


Fig. 11. Distribution for commit size in terms of lines and files by type of contribution.

Figure 11 shows the distribution of the commit sizes for the three types of commits. We clearly see that across all three metrics, bots make smaller contributions than humans, whereas coding agents make larger ones. For instance, the median number of added lines for a human contribution is 11, while for a bot, it is only 4; on the other hand, for AI-assisted commits, it rises to 31, a value triple the median for humans, and closer to the third quartile of human contributions (41). The third quartile of AI-assisted contributions is considerably larger (114).

For deleted lines, the differences are not as stark: we see that the amount of deleted lines per commits for bots is smaller than the other two categories (median: 3), while AI-assisted contributions are larger (median: 7) than human contributions (median: 5). The same is true at the file level. However, the metric being coarser, the differences are smaller: the median bot commit involves a

single file, while both AI-assisted and human commits involve two files. Only in the upper part of the distribution do we see a difference (Q3 for humans: 3; for AI-assisted commits: 4).

Another way to look at the data is to categorize it into discrete size categories. Figure 12 shows the difference in the proportion of size categories for commits, focusing on human-authored and AI-assisted commits. The figure shows that the trend observed in Figure 11 is even more true at the edges of the distribution.

For line additions, the smallest category of commits (adding one to 5 lines) is 36% smaller when they are AI-assisted. On the other hand, the largest commits are almost twice as frequent for AI-assisted commits; this holds true even for commits that add more than 1,000 lines.

We see the same trend, albeit less pronounced, for line deletions: smaller commits (1 to 5 deleted lines) are 12% less frequent in AI-assisted commits, while the larger commits (thousands of deleted lines) are 36% more frequent. Files follow the same pattern, with single-file AI-assisted commits being close to 15% less frequent, while commits with more than 20 files are 30% more frequent.

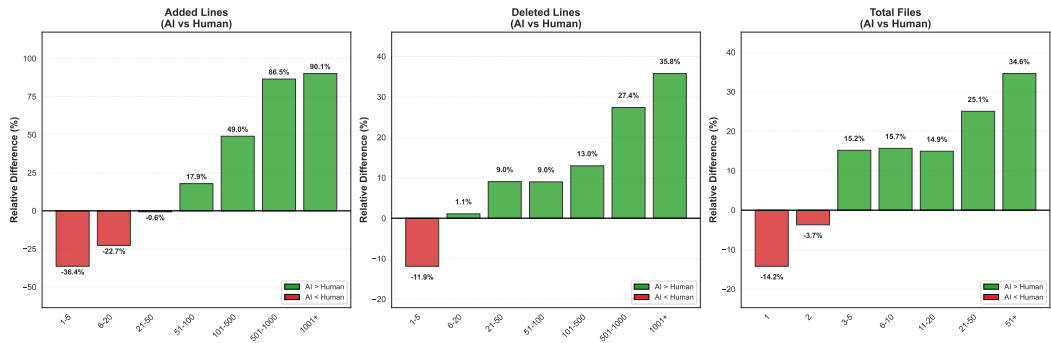


Fig. 12. Comparison between the frequency among size categories of human-authored and AI-assisted commits (lines added, lines deleted, and files involved).

In summary, AI-assisted commits tend to add more lines, in more files, than human-authored commits. In addition, and perhaps somewhat surprisingly, they also delete more code than human-authored commits. Taken together, these trends point towards an increased churn for AI-assisted commits, and for commits changing multiple files with increased frequency. This increased churn and non-local changes, in turn, brings questions on the sustainability of coding agents over time: further study is necessary to evaluate their impact on code quality.

10 RQ6: Types of commits authored by coding agents

As semi-autonomous development assistants, coding agents can contribute in various ways to code bases and support developers across multiple aspects of software development: fix bugs, write documentation, refactor code, write test cases, add new features, etc. Starting from the observation that the adoption of coding agents has surged, with this research question we now focus on *which* tasks agents perform: *What kind of contributions, visible in version control system histories, do developers perform with the help of coding agents?*

To answer this question, we qualitatively examine concrete contributions, at the granularity of individual commits, made by or with agents in git repositories that have adopted them, to better understand the tasks they are used for.

Table 6. Descriptive statistics of the random sample of 790 commits authored by Claude Code, qualitatively analyzed to determine their type.

Metric	Min.	Med.	Avg.	Max.	Type of changed files	Commit count	Language of changed files	Commit count (top-10)
Lines added	0	70	3743.0	1.6 M	source code	573	Markdown	254
Lines deleted	0	9	909.0	576 K	documentation	296	TypeScript	213
Files added	0	1	18.2	10.7 K	data	287	XML	106
Files modified	0	1	3.7	138	configuration	202	JSON	105
Files deleted	0	0	3.0	1959	other	68	Python	102
					build	17	YAML	92
					image	4	JavaScript	73
							Rust	57
							Go	35
							TOML	33

10.1 Dataset

Given the sheer amount of agent-authored commits in our adoption data and the cost of qualitative analysis, we resort to studying a random sample of commit-level contributions. Among the coding agents identified in Section 8, Claude Code leads in popularity (see Figure 8), which makes it a good candidate for qualitative analyzes. Claude Code also defaults to annotating its commits with the Co-authored-by trailer, making the identification of its contributions at the commit level both practical and robust.¹⁶

From the 90,321 commits identified as Claude-authored in our adoption dataset, we draw a sample of 790 for further qualitative analysis using Cochran’s formula for categorical classification into seven categories with a 5% margin of error and a 95% confidence level [10]. We then adjust for the finite population size. As our previous test used a confidence level of 99% and ran 9 statistical tests with p-values $< 10^{-16}$, we apply Bonferroni’s correction [31] to adjust the interval from 95% to 96%, maintaining an overall 95% probability that all our results are correct. These calculations yield a required sample size of 759 commits; we analyze 790 to be conservative. For each commit, we extract the first line of the commit message, diff statistics (added and deleted lines and files), the kind of files impacted by the changes (source code, documentation, configuration, data), and the programming languages used in impacted files.

Table 6 shows some descriptive statistics of the sample.

10.2 Protocol

As manually inspecting the code changes in a large set of commits is impractical (4,652 modified lines per commit on average in our sample), we classify commits based on the content of their messages. In particular, we rely on the Conventional Commits specification,¹⁷ a convention that has seen growing adoption over the past years which encodes the nature of the changes made in a commit within the commit message itself (e.g. *feat* for new features, *fix* for bug corrections, *docs* for documentation) [39]. Originally designed to ease semantic versioning, it defines two core categories: *fix* for bug patching (corresponding to *patch* version increments) and *feat* for new features (corresponding to *minor* version increments). In practice, this set of categories is open-ended and a popular extension is that of semantic commits, which defines seven types of tasks: chore, docs, feat, fix, refactor, style, and test.¹⁸ A commit following that convention has a message prefixed with the name of one of these categories, followed by a colon (e.g. feat: Enable

¹⁶<https://docs.claude.com/en/docs/claude-code/settings>

¹⁷<https://www.conventionalcommits.org/en/v1.0.0/>

¹⁸<https://gist.github.com/joshbucha/6f47e86d2510bce28f8e7f42ae84c716>

Table 7. Types of commits co-authored by Claude Code, for a random sample of 790 commits. Commit types are from the semantic commit messages convention. Commit classification performed by either parsing the first line of commit messages according to the Conventional Commits specification, version 1.0.0 (for the 513 commits that conform to it) or via manual review performed by two authors independently (277 commits).

Type	Count	Ratio (%)
feat	282	35.7%
fix	236	29.9%
docs	86	10.9%
refactor	78	9.9%
chore	56	7.1%
test	43	5.4%
style	9	1.1%
Total	790	

filtering clients per organization). This convention enables a structured and scalable way to infer the intent and type of work represented by each commit without examining the underlying code. By leveraging this convention, we can analyze contribution patterns systematically and with reasonable confidence in the accuracy of the classification.

To our surprise, the majority of Claude commits in our sample already follows this convention: 513 of the 790 commits (65%) can automatically be classified into one of the seven categories by matching their prefix. Indeed, coding agents may imitate the style of commit messages made by developers (in the current project or in their training data), or be explicitly instructed to follow that specific convention. When a commit can be classified automatically, we retain the classification made by the agent or developer who made the commit. Then, two authors were tasked to manually and individually classify the remaining 277 commits into one of the seven categories by inspecting their commit message and diff, with an initial agreement on 250 of the 277 commits (90%). The remaining 27 commits were finally classified after discussion.

10.3 Results

In a recent study, Zeng et al. analyze 88,704 *human-authored* commit messages in 116 repositories from diverse domains following the Conventional Commit specification. They find that *chore* commits are the most frequent (non-functional changes, including updating build scripts, dependencies, and continuous integration—31%), followed by *fix* (bug fixes—27%) and *feat* (implementation of new features—17%) [39].

This is in stark contrast with our sample of Claude commits, as shown in Table 7. Commits of types *feat* and *fix* are the most frequent in our sample, accounting for two thirds of the analyzed commits, while *chore* commits only account for 7.1%. Notably, feature-introducing commits are about twice as common for agents as for humans. This likely contributes to the pattern observed in Section 9 that AI-assisted commits tend to modify more lines and touch more files than human-authored ones. For the remaining categories, the percentages differ by two percent or less between the study of Zeng et al. and ours, reflecting a similar distribution for these tasks between humans and agents.

Several factors might explain these differences. As shown in Table 3, while agent adoption spans the whole spectrum, it is the highest in younger projects, where the implementation of new features might be prioritized over bug fixing and maintenance tasks such as *chore*. Besides, several *chore* tasks are already handled well by automated bots (e.g. dependabot for dependency management),

so the cost of running agents might offset their benefits in this situation. The low number of *test* commits doesn't necessarily mean agents aren't being used for testing. The Conventional Commit specification requires choosing a single category, so tests added alongside a new feature might be included in a *feat* commit instead. For example, commit [d8d58f61](#) in `nizos/tdd-guard` (*feat*: add AnthropicModelClient with API key configuration) introduces a new feature together with corresponding test cases and is classified as a *feat* commit. The proportion of standalone *test* commits is similar to the proportion found by Zeng et al. [39].

Moreover, in line with the observation on contribution size, test commits, as well as other categories such as refactorings and documentation, despite being less common, can be quite large. For instance, commit [4a9dd8f](#) adds extensive unit tests for two components (two files added, +872 lines of code); commit [3bcefd2](#) is a large refactoring that changes the structure of the code (and also adds some tests, 14 files changed; +912/-19 lines); commit [f6315ff](#) adds a high-level documentation of the structure of a module written in Zig, describing components, providing examples of use, with specific details on performance (one new file, 180 lines).

In summary, agents are used for a wide range of software development tasks, spanning the full set of categories defined by the Conventional Commits specification. Compared with humans, however, agents are used more often to introduce new features and less often for long-term maintenance tasks such as dependency and continuous integration management. These observations align with our previous findings about the age of agent-adopting projects and the size of the contributions attributed to them.

11 Discussion

We first discuss the limitations of our study, focusing in particular on the ways our study could either under-estimate or over-estimate adoption; we conclude that an under-estimate is more likely. We then discuss the implications of this work for practitioners and researchers, keeping in mind that it is likely we are under-approximating coding agent adoption.

11.1 Limitations

Generalizability of our findings. We took several precautions to maximize the external validity of our findings. We purposefully selected a large dataset of GitHub projects, selecting for projects that reach minimum size and activity requirements (5,000 lines of code; 100 commits). These minimum size and activity requirements are high enough to ensure we select projects that are more likely to reflect real programming activity, but low enough to increase diversity (e.g., including both small and large projects, projects with few and many contributors, that span many topics and programming languages). Further, after estimating adoption as a whole in RQ1, in RQ2 we analyzed our adoption metrics by relating them to other project characteristics (size; age; contributors; and activity indicators: commits, issues, pull requests) in order to better understand how adoption varied with these characteristics. However, we restricted these analyses to high-level metrics only, as carrying out similar analyses in the other RQs would have significantly lengthened the paper.

Similarly, we opted to exclude forks to avoid including copies of seemingly large projects, but with possibly limited activity. Finally, to increase the relevance of our findings to real-world coding scenarios, we specifically looked at the adoption from repositories of specific, established organizations.

Despite all of these mitigations, it is possible that our results do not translate well to more general use, and to industry in particular: we have reasons to think that industry use might be different than use in open-source, which we detail at the end of Section 11.3; we think it is likely that the use in industry is higher than in open-source. More generally, we have identified several reasons why our specific metrics could lead us to either *over-estimate* the adoption and use of coding agents,

or on the contrary, to *under-estimate* it. We expand on these themes below. When relevant, we mention some of the perils of agent mining, that we identified and developed in another work [28]. In particular, many of the reasons for under or over-approximating agent activity stem from the Peril of Partial Observability.

Other limitations. The commit ratio, for reasons explained earlier, is an imperfect metric. It is particularly the case at the individual project level, where the observability depends on the choices of individual contributors. This is why we refrain from studying projects in isolation. We assume that these kind of errors are evenly distributed when analyzing multiple projects, but we can not guarantee it. In particular, populations with a higher knowledge of coding agents may behave differently.

For the label analysis in RQ3, we are limited by the labels chosen by developers. A small minority of projects have such labels; moreover, many labels co-occur, which adds some degree of correlation between individual labels.

For the programming language analysis, we rely on GitHub Linguist. This is a proven solution, but in ideal scenarios, it uses both file names and file contents. This would not scale to our study, as we apply it to all committed files in 2025 for 128,018 projects; we limit GitHub linguist to file names and extensions, which makes it less precise. For instance, PHP is misclassified as one of its dialects, Hack. We tried to resolve the most important issues (i.e., Rust), but can not guarantee we found all the errors.

Pull requests that are “squash merged” condense multiple commits in a single one [5]. If parts of the commits in the PR was AI-assisted, we are forced to label the entire merge commit as AI-assisted. This may lead us to over-estimate the size of the AI contributions when the proportion of AI-assisted commits in the PR is small. Aside from analyzing the diff, which is difficult at the scale of the analysis, the other option left would be to ignore these commits, which we found to be a worse tradeoff.

11.2 Reasons for over-estimating coding agent use

We first focus on the reasons why we could over-estimate adoption.

Under-use of agents. A large proportion of projects have evidence of coding agent use at the file level, but none at the commit level: only 61.03% of projects do; moreover, Figure 3 shows that the correlation between the metrics is very weak. This is a manifestation of the Peril of Partial Observability, and we can not know to which extent agents are used in these projects. It is very likely that, after early experiments, some of these projects are *not* using coding agents, but we do not know the proportion.

Extent of the involvement of the agent. When we detect agentic activity, we can never be sure of the proportion of the activity that is due to the agent, due to the Peril of Partial Observability. In general, we can only observe the end result of the work (commits or pull requests). For a commit authored or co-authored by an agent, the involvement of the developer may range from the minimal (e.g. “vibe-coding”, or trusting the agent enough to give it extensive permissions and autonomy), to a very close oversight, closely reading the code, or even significantly editing it.

Similarly, when we identify agent activity in a pull request for an agent that does not sign their commit (such as Codex), we assume that all the commits from the initial author stem from the agent. However it is possible that only part of the commits were done by the agent, and follow-up work was performed by the developer. The Peril of Diversity [28] implies that there is a diversity of agent workflows, which makes tailoring heuristics for this difficult.

Size is not everything. In RQ5, we compare the size of the commits, finding that AI-assisted commits are larger in terms of lines added, lines deleted, and files involved. How this affects productivity is an open question, which we do not try to answer (we leave that to future work). For instance, the data is consistent with a high churn (lines added and quickly deleted), which might not translate to real productivity. Similarly, our analysis does not look at whether AI-assisted commits are more likely to be reverted (which we have seen on occasion), whether in other AI-assisted commits, or by humans. Computing churn in this way would require running origin analyses [9] at scale (particularly in case of extensive refactorings [15]), which is a significant endeavor.

11.3 Reasons for under-estimating coding agent use

On the other hand, there are several reasons why we may *under-estimate* the use of coding agents.

Absence of evidence is not evidence of absence. The flipside of the Peril of Partial Observability is that, for the projects in which we have no evidence of use (either at the commit level, or at the file level), there might very well be *significant* use of coding agents. First, there is the simple fact that some agents simply do not co-author commits; this is an implementation choice in the harness. For instance, Codex does not sign its commits, partly because its workflow is more based on pull requests. Second, if a coding agent signs their commits or leaves traces in pull requests, there can be ways to disable this behavior. For instance, there is a boolean setting to toggle this in Claude Code; and there is a similar setting to set the branch prefix that Codex uses. This makes it very easy to make their activity undetectable. Anecdotally, we have seen developers go to considerable lengths to disable this behavior even without using these settings, such as including extensive guidance instructing Claude Code not to sign commits, or setting up a GitHub action to auto-remove labels inserted by Codex. Finally, developers wishing more oversight may integrate agents in their workflow, but prefer to commit manually, after checking the agent's contributions; we suspect this is a sizeable portion.

In addition, we rely on the presence of files to scale our search for coding agent usage, but we have also found that as many projects, if not more, have visible coding agent activity only in commits. Once again, this depends on both the coding agent and the developer workflow. Some agents, such as Devin, work primarily using a cloud setup, where they clone the project and handle their state without committing to the repository. Other agents such as Lovable or Vercel v0¹⁹ are dedicated to fast prototyping, starting from an installation in the cloud, and as such are under-represented in Github repositories.

Some developers will avoid to have configuration files in their GitHub repositories, using specialized dotfiles repositories. Indeed we have several hundred such dotfiles repositories in our dataset (which we filter out to estimate overall adoption). To further improve our coverage of projects, we could check for the presence of agent configuration files in a dotfiles repository, and link it to other repositories the developer contributes in. However, this would significantly complexify our mining pipeline, and it would add noise: not every developer has a public dotfiles repository, and their agent use could be confined to a few of their repositories.

Missing heuristics. There is a large number of coding agents (Peril of Multiplicity [28]), which is increasing, and the agents themselves are evolving (Peril of Velocity). Keeping track of all the coding agents, and finding appropriate heuristics for all of them, is a significant endeavor (rendered more complex by the Peril of Diversity: each agent works differently). We tried to be as thorough as possible, but we have almost certainly missed some agents, or at least some heuristics for a given agent. This will lead us to under-estimate the use of these specific agents. Related to this is the fact

¹⁹Lovable: <https://lovable.dev/>; Vercel v0: <https://v0.app>

that some agents can use the guidance of others, making the identification of individual agents difficult. This is why, for instance, we can not attribute with 100% certainty all uses of AGENTS.md to Codex.

Unmerged commits. Our analysis focuses on the commits that are merged in the main branch. There is more use of coding agents in unmerged Pull Request and branches, which we do not consider as we focus on the contributions that end up in the main version of the project. These PRs can be unmerged for a variety of reasons: e.g., they are very recent and have not been reviewed yet; or, the agent's work was subpar. We have occasionally seen pull requests that have been merged in non-conventional ways (e.g. manually), which our study can not detect. In addition, unmerged PRs can represent proof of concept or prototyping work, which might be useful even if the actual code does not last.

Things may be different in the industry. While it is well-known that studying open-source software is an imperfect proxy for industry software, it is particularly so for coding agents, due to their cost (Peril of Costs Shape Usage [28]). Coding agents rely on long LLM inferences, over multiple turns, which is expensive. As a consequence, using coding agents most often requires a subscription, or metered API usage; free usage is limited to less powerful models and stricter rate limits. This means that industry developers may use coding agents more as part of a paid job (especially if it is subsidized by their employer), while open-source developers, who may not have such ease of access, may limit their usage. In addition, beyond the amount of usage, the *kind* of usages made possible by less restrained use may be different (e.g., using multiple agents in parallel). Anecdotally, developers running high coding agent bills (hundreds of dollars per month or more) are not unheard of, while some are curious about the possibilities offered by spending even more²⁰. Thus, it is possible that the usage in industry is higher than in open-source. We tried to look into this by studying specific organizations, finding that they had higher file-level adoption in RQ3 (a relative increase of $\approx 42\%$). However, since this is only based on public repositories, it likely does not reflect the internal usage.

11.4 Putting it all together

Considering all the reasons for undercounting and overcounting AI adoption, we think it is more likely that we are undercounting it. First, we think that projects adopting coding agents at the file level that have no AI-assisted commits are more likely to be using coding agents, rather than having abandoned them. This is because it is very easy to configure the main coding agents not to leave traces in commits or PRs, and because, for now, we can not account for developers that commit manually yet still use coding agents. While this is only anecdotal, there seem to be a non negligible number of projects that fit these cases²¹. While it is conceivable that projects with very limited and unchanging guidance have abandoned coding agents, it is less likely that projects with extensive and/or actively maintained guidance have done so.

We did a preliminary analysis of a related issue: after an initial usage, some projects experience a fall of their commit ratio to 0. A straightforward interpretation of this would be that such a project has abandoned coding agents. However, we found that on a sample of such projects (the 500 projects that had the largest commit ratio drop to zero), half of them had added or modified at least one agent guidance or configuration file after the commit ratio dropped to 0. It seems unlikely that a project that has abandoned coding agents would continue maintaining their coding agent guidance.

²⁰For instance, this Shopify executive wondering how a developer may spend ca. 10,000 dollars a month on coding agents <https://xcancel.com/fnthawar/status/1930367595670274058>

²¹For some clear cut cases that our heuristics nevertheless would miss, see the following examples: [Port of Graph::Easy library to TypeScript](#) (evidence: [blog post](#)) and [AWK Programming language dialect](#) (evidence: repository README file).

Table 8. Main findings of our six research questions on the adoption of coding agents, based on analyzing 128,018 GitHub repositories.

Research question	Summary
RQ1: Overall adoption	As of February 21st, 2026, 22.20% to 28.66% of our dataset adopted coding agents.
RQ2: Adoption and metrics	File-level adoption is higher for both <i>younger</i> and <i>larger</i> projects; larger projects have more <i>experimental</i> and less <i>pervasive</i> commit-level adoption.
RQ3: Contexts of adoption	Adoption is $\approx 42\%$ larger in the top 20 organizations; it is broad in terms of languages and topics.
RQ4: Evolution	Adoption is increasing over time, with a last acceleration in early 2026.
RQ5: Contribution size	AI-assisted commits are <i>larger</i> than human-authored commits.
RQ6: Contribution types	Most AI-assisted commits are <i>feature additions</i> or <i>bug fixes</i> .

As another datapoint that fits with this narrative, consider Figure 5, in which some topics such as agents, llm, or openai, have low to average commit ratios, despite these topics showing very high file-level adoption in Figure 4. Rather than having a lower than average level of adoption, we hypothesize it is more likely that contributors to these projects, having more knowledge of coding agents, are more likely to be aware of how to change their settings and disable the visibility of their activity.

Second, given the high ratio of file-level activity that has no commit level activity, we think there is a sizeable proportion of “undetected use” of coding agents: projects that have neither file-level nor commit-level agent activity, yet still use agents. For instance, the projects mentioned in this earlier footnote²¹ fit this profile, making us believe that the true adoption is higher than our conservative estimate. To be clear, this is so far only a hypothesis; future work should define heuristics to detect coding agent activity in these (and more challenging cases) with sufficient precision. Only such a study could determine with more certainty where the true adoption lies between our conservative and our high estimate, and could provide a more precise estimate of the AI-assisted commit ratio in adopting projects.

Finally, the third way adoption could be higher is the difference between open-source and industry, for the reasons mentioned above. Taken together, we think that these three factors make it more likely that: 1) for a sizeable subset of projects where we have detected activity, the true commit ratio is significantly higher than what we can measure; 2) some projects we classify as non adopters are actually adopters; and 3) adoption in private repositories from industry (on GitHub and elsewhere) is likely higher than OSS. While on the other hand, there is certainly a portion of projects that do have abandoned coding agents, we think the balance weights towards higher, rather than lesser adoption.

11.5 Implications

We recall the main findings of our study (see Table 8), before discussing their implications.

The adoption of coding agents is fast, and broad. According to RQ1, our conservative estimate of coding agents is 22.20%, as of February 21st, 2026 while our high estimate is 28.66%. As mentioned earlier (footnote ¹), the most recent data puts the estimate between 31.63% and 38.23% [29]. This is a technology that saw the light of day in 2024; it was confidential in 2024 and before the spring of 2025, when increase in model capabilities and the release of coding agents by major actors coincided with a drastic increase in adoption rate.

Moreover, if there are variations in adoption intensity in terms of project characteristics, project topic, or programming language, we can see that the adoption is *broad*:

- Adoption is not especially concentrated on smaller projects (be it measured in lines of code, contributors or commits). File-level adoption rather suggests the opposite, and commit-level adoption, even if less strong on such larger projects, is far from insignificant (“Pervasive” adopters are decreasing with size, but “Consistent” adopters are stable). Moreover, important organizations have high adoption of coding agents. There is, however, a bias towards younger projects.
- Likewise, adoption is not especially concentrated on particular types of projects. Some categories of projects may be more or less popular than the average, but there does not appear to be many categories that have near-zero adoption. This holds for both file-level and commit-level adoption. At the file level, some categories of projects are seeing much larger adoption, particularly in topics close to AI (a sort of “dogfooding” effect?), however, at the commit level even these categories see comparable usage to the rest.
- Finally, adoption is not especially concentrated on the small set of the most popular programming languages. Some less common languages (e.g., Dart, Ruby, Swift, PHP) see similar commit-level adoption to major languages. However, data is limited for the truly uncommon languages.

Coding agents are a step change from LLM-based code completion. Looking at the results of RQ5 and RQ6, we can infer that coding agents go beyond traditional LLM-based code completion. While LLM-based code completion helped developers write lines or blocks of code, the scope of coding agents is larger. This can be seen just by the size of AI contributions, which is clearly larger than human contributions: the median lines of code in an AI-assisted commit is triple that of human-authored ones; the proportion of very large AI-assisted commits (500 or more lines of code, or even more than 1000) is twice as frequent. AI-assisted commits are also larger in terms of deleted lines, and involved files. In addition, the type of contributions is telling: the most common type of contributions that we see in RQ6 are the implementation of *features*, indicating that coding agents are used in tasks spanning a relatively wide scope.

Note that, if, as we suspect, we are missing a sizeable proportion of AI-assisted commits, it is likely that the difference between the human and AI-assisted commits is *larger*, since those undetected commits are at the moment classified as human-authored commits.

Performance Outlook. If, as the adoption curve in RQ4 suggest, this trend continues, we can expect coding agents to become extremely common in 2026 or 2027. The updated version of this study does not point to a slowdown yet (as of early June 2026 [29]). In fact, a version of this study, with an identical pipeline, but conducted on 12,794 GitHub projects created between 29/08/2025 and 04/04/2026 (and ran on 04/04/2026) finds a drastically higher adoption rate, from 71.83% to 76.15% [30]. It also finds that adoption intensity (measured via the commit ratio) is much higher in the newer projects.

This is even more the case, if, as we hypothesize, we are under-estimating, rather than over-estimating, the adoption of coding agents. Related to this is the increase of capabilities of the underlying models: much as stronger models were a contributing factor to the adoption of coding agents in 2025, similar increases could drive the adoption further in 2026 and beyond. The Anthropic economic index [1] finds that a growing number of tasks are delegated to LLMs, pointing at increased autonomy. Similarly, recent work [18] looks at 50% and 80%-task-completion time horizon for agents (the time humans typically take to complete tasks that AI models can complete with 50%/80% success rate). They observe that, historically, the task completion time has been doubling every 7

months. This increased autonomy, due to evolving model capabilities, is consistent with the way coding agents work in the ideal cases, and is consistent with the increased adoption we observe in Figure 7.

The use of coding agents in practice should be studied extensively. Taken together, the previous three observations come to an important implication. Coding agents, making large contributions are adopted quickly, broadly and sometimes very extensively (a small but non-negligible proportion of projects have a very high ratio of AI-assisted commits). We have reasons to expect this trend to continue further. Therefore, it is *extremely important* to study their impact in practice, in order to provide actionable advice to practitioners.

Since the emergent of practical coding agents is very new, the body of work so far is very limited (see Section 2). Given the speed of the phenomenon, we argue that the research community should study this with urgency; we hope that the initial strides we made in this field will be helpful for others²². While many aspects can be studied, we have identified a few we wish to highlight:

- While the large contributions of coding agents can be seen as an indicator of productivity, the large amount of deleted code is an indicator of churn, which has been traditionally associated with defects [25]. It is thus important to investigate to which extent coding agents cause high code churn, and what its consequences are. There is already evidence that AI-assisted development in general contributes to high code churn [11, 12]. In addition, recent work points at emerging quality issues associated with the use of coding agents [13].
- On the other hand, perhaps such a high churn is not always problematic. Coding agents, by making code extremely cheap to write, can enable scenarios that were not possible before (e.g., prototyping multiple variants of a solution to a problem, before making a more informed decision). In essence, coding agents greatly facilitate the creation of disposable, exploratory code, which may have a very short, but very useful, life. Characterizing these scenarios would be valuable future work.
- Regardless of its quality, the amount of code produced is already putting strains on the system, to the point that some projects change their contribution guidelines in reaction [14]. Studies of the impact of coding agents on development communities also constitute an important area of future work.
- More generally, we think it is valuable to study the minority of extreme adopters of coding agents (those that we identify as having very high commit ratios). These adopters may give us early signals that could be useful to the broader community. They might act as “canaries in a coal mine”, or cautionary tales for prospective users of coding agents. On the other hand, they might help discover new usages of coding agents that may be useful to practitioners.
- An essential component of coding agents is their guidance. As we have seen in RQ2, the amount of guidance used, and its update frequency, varies by several orders of magnitude, from the most basic to the most sophisticated. Detailed studies of this guidance, what it contains, and how it is used, will be essential to better understand how it impacts the behavior of coding agents. There is very early work in classifying the type of guidance [22].
- Last but not least, underlying all of this is the identification of coding agent activity that we are not able to detect at the moment. There is considerable future work in devising heuristics to detect this activity, and strengthen the body of work that will be based on this.

A call for clear attribution. To study the use of coding agents in the best conditions, proper identification is essential. While we think detecting agent activity is an important avenue for future research, an even more effective solution would be to reliably document it. This is why we call

²²We share all the datasets and analyses in our replication package to facilitate this.

on the practitioners to document, rather than hide, their use of coding agents. We also call on providers of coding agents to continue their standardization efforts, while defining mechanisms to identify individual coding agents.

12 Conclusion

Coding agents based on LLMs appeared in 2024, and took off in 2025. This work presented a study of the adoption of coding agents in GitHub projects. We find that, as of February 21st, 2026, between 22.20% and 28.66% of projects in a large dataset of 128,018 GitHub projects show traces of use of coding agents. We find that this adoption was fast (it happened mainly from March to October 2025), broad (it includes all kinds of projects, in a variety of programming languages and organizations), and is still increasing [29, 30]. At the commit level, we find that a sizeable proportion of projects exhibit “Consistent” (5 to 20%) or even “Pervasive” use (more than 20%); a small, but non-negligible proportion of projects show extreme use. Moreover, we find that coding agents are used to implement features and bugs fixes, and do so via large contributions (larger than humans). These findings have implications both for the practice, which is seeing a very rapid transition towards coding agents, and for researchers, which should study this phenomenon in order to provide advice to practitioners during this transition.

Acknowledgments

This study received financial support from the French State (Investments for the Future programme, IdEx université de Bordeaux).

References

- [1] Anthropic. 2025. *Anthropic Economic Index: Uneven Geographic and Enterprise AI Adoption*. Technical Report. Anthropic PBC. <https://assets.anthropic.com/m/218c82b858610fac/original/Economic-Index.pdf> Report, 15 Sept 2025.
- [2] Shraddha Barke, Michael B James, and Nadia Polikarpova. 2023. Grounded copilot: How programmers interact with code-generating models. *Proceedings of the ACM on Programming Languages* 7, OOPSLA1 (2023), 85–111.
- [3] Joel Becker, Nate Rush, Elizabeth Barnes, and David Rein. 2025. Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity. *arXiv preprint arXiv:2507.09089* (2025).
- [4] Joel Becker, Nate Rush, Tom Cunningham, David Rein, and Khalid Mahamud. 2026. We are Changing our Developer Productivity Experiment Design. <https://metr.org/blog/2026-02-24-uplift-update/>.
- [5] Christian Bird, Peter C Rigby, Earl T Barr, David J Hamilton, Daniel M German, and Prem Devanbu. 2009. The promises and perils of mining git. In *2009 6th IEEE International Working Conference on Mining Software Repositories*. IEEE, 1–10.
- [6] Islem Bouzenia and Michael Pradel. 2025. Understanding Software Engineering Agents: A Study of Thought-Action-Result Trajectories. *arXiv preprint arXiv:2506.18824* (2025).
- [7] Roberto Di Cosmo and Stefano Zacchiroli. 2017. Software Heritage: Why and How to Preserve Software Source Code. In *Proceedings of the 14th International Conference on Digital Preservation (iPRES 2017)*. ACM, 1–10.
- [8] Ozren Dabic, Emad Aghajani, and Gabriele Bavota. 2021. Sampling projects in github for MSR studies. In *2021 IEEE/ACM 18th International Conference on Mining Software Repositories (MSR)*. IEEE, 560–564.
- [9] Michael W Godfrey and Lijie Zou. 2005. Using origin analysis to detect merging and splitting of source code entities. *IEEE Transactions on Software Engineering* 31, 2 (2005), 166–181.
- [10] Leo A Goodman. 1965. On simultaneous confidence intervals for multinomial proportions. *Technometrics* 7, 2 (1965), 247–254.
- [11] William Harding. 2025. AI Copilot Code Quality: Evaluating 2024’s Increased Defect Rate via Code Quality Metrics. https://www.gitclear.com/ai_assistant_code_quality_2025_research Accessed on October 13th, 2025.
- [12] William Harding and Matthew Kloster. 2024. Coding on Copilot: 2023 Data Suggests Downward Pressure on Code Quality. https://www.gitclear.com/coding_on_copilot_data_shows_ais_downward_pressure_on_code_quality Accessed on 03 24, 2024.
- [13] Hao He, Courtney Miller, Shyam Agarwal, Christian Kästner, and Bogdan Vasilescu. 2025. Speed at the Cost of Quality? The Impact of LLM Agent Assistance on Software Development. *arXiv preprint arXiv:2511.04427* (2025).
- [14] Andre Hora and Romain Robbes. 2026. AI Policy, Disclosure, and Human in the Loop: How Are Contribution Guidelines Adapting to GenAI? *arXiv preprint arXiv:2605.16706* (2026).

- [15] Andre Hora, Danilo Silva, Marco Tulio Valente, and Romain Robbes. 2018. Assessing the threat of untracked changes in software evolution. In *Proceedings of the 40th International Conference on Software Engineering*. 1102–1113.
- [16] Saki Imai. 2022. Is github copilot a substitute for human pair-programming? an empirical study. In *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings*. 319–321.
- [17] Aayush Kumar, Yasharth Bajpai, Sumit Gulwani, Gustavo Soares, and Emerson Murphy-Hill. 2025. Sharp Tools: How Developers Wield Agentic AI in Real Software Engineering Tasks. *arXiv e-prints* (2025), arXiv–2506.
- [18] Thomas Kwa, Ben West, Joel Becker, Amy Deng, Katharyn Garcia, Max Hasin, Sami Jawhar, Megan Kinniment, Nate Rush, Sydney Von Arx, et al. 2025. Measuring ai ability to complete long tasks. *arXiv preprint arXiv:2503.14499* (2025).
- [19] Alexander Lex, Nils Gehlenborg, Hendrik Strobelt, Romain Vuillemot, and Hanspeter Pfister. 2014. UpSet: Visualization of intersecting sets. *IEEE Transactions on Visualization and Computer Graphics* 20, 12 (2014), 1983–1992.
- [20] Hao Li, Haoxiang Zhang, and Ahmed E. Hassan. 2025. AIDev: Studying AI Coding Agents on GitHub. <https://doi.org/10.5281/zenodo.16919051>. Accessed 2025-10-21.
- [21] Jenny T Liang, Chenyang Yang, and Brad A Myers. 2024. A large-scale survey on the usability of ai programming assistants: Successes and challenges. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*. 1–13.
- [22] Seyedmoein Mohsenimofidi, Matthias Galster, Christoph Treude, and Sebastian Baltes. 2025. Context Engineering for AI Agents in Open-Source Software. *arXiv preprint arXiv:2510.21413* (2025).
- [23] Hussein Mozannar, Gagan Bansal, Adam Fournay, and Eric Horvitz. 2022. Reading between the lines: Modeling user behavior and costs in AI-assisted programming. *arXiv preprint arXiv:2210.14306* (2022).
- [24] Vijayaraghavan Murali, Chandra Maddila, Imad Ahmad, Michael Bolin, Daniel Cheng, Negar Ghorbani, Renuka Fernandez, and Nachiappan Nagappan. 2023. CodeCompose: A large-scale industrial deployment of AI-assisted code authoring. *arXiv preprint arXiv:2305.12050* (2023).
- [25] Nachiappan Nagappan and Thomas Ball. 2005. Use of relative code churn measures to predict system defect density. In *Proceedings of the 27th international conference on Software engineering*. 284–292.
- [26] Addy Osmani. 2025. The reality of AI-Assisted software engineering productivity. Substack blog post. <https://addy.substack.com/p/the-reality-of-ai-assisted-software-engineering-productivity> “What the data really shows about AI coding tools in 2025.”
- [27] Sida Peng, Eirini Kalliamvakou, Peter Cihon, and Mert Demirel. 2023. The impact of ai on developer productivity: Evidence from github copilot. *arXiv preprint arXiv:2302.06590* (2023).
- [28] Romain Robbes, Théo Matricon, Thomas Degueule, Andre Hora, and Stefano Zacchiroli. 2025. Promises, Perils, and (Timely) Heuristics for Mining Coding Agent Activity. *Accepted for publication at MSR 2026* (2025).
- [29] Romain Robbes, Théo Matricon, Thomas Degueule, Andre Hora, and Stefano Zacchiroli. 2026. Agentic Much? Adoption of Coding Agents on GitHub. *arXiv preprint arXiv:2601.18341* (2026).
- [30] Romain Robbes, Théo Matricon, Thomas Degueule, Andre Hora, and Stefano Zacchiroli. 2026. Agentic Very Much! Adoption of Coding Agent in New GitHub Projects. *arXiv preprint arXiv:2606.07448* (2026).
- [31] G Rupert Jr et al. 2012. Simultaneous statistical inference. (2012).
- [32] Klaas-Jan Stol and Brian Fitzgerald. 2018. The ABC of software engineering research. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 27, 3 (2018), 1–51.
- [33] Mario F Triola, William Martin Goodman, Richard Law, and Gerry Labute. 2006. *Elementary statistics*. Pearson/Addison-Wesley Reading, MA.
- [34] Rosalia Tufano, Antonio Mastropaolo, Federica Pepe, Ozren Dabić, Massimiliano Di Penta, and Gabriele Bavota. 2024. Unveiling ChatGPT’s Usage in Open Source Projects: A Mining-based Study. *arXiv preprint arXiv:2402.16480* (2024).
- [35] Edward R. Tufte. 2004. Sparkline Theory and Practice. Online article. <https://www.edwardtufte.com/notebook/sparkline-theory-and-practice-edward-tufte/> “A small, intense, word-sized graphic with typographic resolution.”
- [36] Priyan Vaithilingam, Tianyi Zhang, and Elena L Glassman. 2022. Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models. In *Chi conference on human factors in computing systems extended abstracts*. 1–7.
- [37] Ruotong Wang, Ruijia Cheng, Denae Ford, and Thomas Zimmermann. 2023. Investigating and designing for trust in ai-powered code generation tools. *arXiv preprint arXiv:2305.11248* (2023).
- [38] Tao Xiao, Youmei Fan, Fabio Calefato, Christoph Treude, Raula Gaikovina Kula, Hideaki Hata, and Sebastian Baltes. 2025. Self-Admitted GenAI Usage in Open-Source Software. *CoRR abs/2507.10422* (2025). arXiv:2507.10422
- [39] Qunhong Zeng, Yuxia Zhang, Zhiqing Qiu, and Hui Liu. 2025. A First Look at Conventional Commits Classification. In *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering*. 2277–2289.
- [40] Albert Ziegler, Eirini Kalliamvakou, X Alice Li, Andrew Rice, Devon Rifkin, Shawn Simister, Ganesh Sittampalam, and Edward Aftandilian. 2022. Productivity assessment of neural code completion. In *Proceedings of the 6th ACM SIGPLAN International Symposium on Machine Programming*. 21–29.

Table 9. List of coding agents with their URLs

Name	URL
Aider	https://aider.chat/
Alibaba Lingma	https://www.alibabacloud.com/en/product/lingma
AmazonQ	https://aws.amazon.com/q/
Amp	https://sourcegraph.com/amp
Atlassian Rovodev	https://www.atlassian.com/software/rovo-dev
Augment Code	https://www.augmentcode.com/
Azad	https://www.azad.bot
Baidu Comate	https://comate.baidu.com/
Brokk	https://brokk.ai/
Charlie	https://www.charlielabs.ai
ChatGPT	https://chatgpt.com/
Claude Code	https://www.claude.com/product/claude-code
Cline	https://cline.bot/
Codebuddy	https://www.codebuddy.ai/
Codegen	https://codegen.com/
Coderabbit	https://coderabbit.ai/
Codex	https://openai.com/codex
Continue	https://www.continue.dev/
Copilot	https://github.com/features/copilot
CopilotSwe	https://github.blog/changelog/2025-09-22-copilot-swe-model-rolling-out-to-visual-studio-code-insiders/
Crush	https://github.com/charmbracelet/crush
Cursor	https://cursor.com/
Deepsource	https://deepsource.com/
Devin	https://devin.ai/
Factory	https://factory.ai/
Factory Droid	https://factory.ai
Fly	https://fly.io/
GPT-Engineer	https://github.com/AntonOsika/gpt-engineer
Gemini	https://gemini.google.com/
Generic	https://agents.md
Goose	https://github.com/block/goose
Gru	https://gru.ai/
Jules	https://jules.google/
Junie	https://www.jetbrains.com/junie/
Kilo Code	https://kilocode.ai/
Kiro	https://kiro.dev/
Langchain Open SWE	https://github.com/langchain-ai/open-swe
Letta Code	https://www.letta.com
Microsoft Amplifier	https://github.com/microsoft/amplifier
Mistral Vibe	https://docs.mistral.ai/mistral-vibe/introduction
Ona	https://ona.com/
OpenHands	https://github.com/All-Hands-AI/OpenHands
Opencode	https://opencode.site
Phoenix	https://phoenix.new/
Pi	https://pi.dev/
Plandex	https://plandex.ai/
Qodo	https://qodo.ai
Qwen Coder	https://github.com/QwenLM/qwen-code
Roo Code	https://github.com/RooCodeInc/Roo-Code
Rulesync	https://github.com/dyoshikawa/rulesync/tree/main
Sentry Seer	https://sentry.io/product/seer
Serena	https://github.com/oraios/serena
Sketch	https://www.sketch.com/
Sourcery	https://sourcery.ai/
SpecKit	https://github.com/github/spec-kit
Sweep	https://sweep.dev/
Taskmaster	https://github.com/eyaltoledano/claude-task-master
Tessl	https://tessl.io
Trae	https://www.trae.ai/
Verdent	https://verdent.ai
Warp	https://www.warp.dev/
Windsurf	https://windsurf.com/
Zed	https://zed.dev/

A Selected coding agents

Table 9 shows the list of coding agents we include in this study.